Chip Design & Verification

Lab 4 - Arad Greenshtein

מטרת המעבדה היא לתכנן ולממש על לוח Nexys A7 בקר שידור UART. המשתמש קובע במתגים תו, זמן השהיה וגודל של ריבוע, ובלחיצה קצרה על הכפתור האמצעי הערכים ננעלים והבקר משדר למחשב ריבוע של N על N פעמים התו, בקצב של baud 57600. בין תו לתו באותה שורה נשלח רווח, בסוף כל שורה נשלחים CR ו-LF, ואחרי כל תו נתונים הבקר ממתין את זמן ההשהיה שנבחר. תצוגת segment-7 מציגה את התו, ההשהיה, הגודל ומספר השורות שכבר נשלחו, ושני לדים מראים את התקדמות השידור. כפתור CPU Reset מאפס את כל המערכת.

Step 1 - Requirements & Design Decisions

Interface

Signal	Direction	Width	Polarity
clk (100MHz)	input	1	-
rst_n (CPU Reset)	input	1	active low
btn_c (Center)	input	1	active high
sw	input	16	active high
uart_tx	output	1	idle high
led_byte (LD0)	output	1	active high
led_done (LD15)	output	1	active high
an	output	8	active low
seg (CA..CG, DP)	output	8	active low

Switch mapping

כל אחד מהשדות מקבל קבוצת מתגים משלו. ארבעה מתגים לא נדרשים בתכן, והסינתזה מסירה אותם.

Switches	Field	Values
sw[7:0]	Character	ASCII code (hex)
sw[9:8]	Delay after each data char	00 = 0, 01 = 50 ms, 10 = 100 ms, 11 = 200 ms
sw[14:13]	Square size N	00 = 1, 01 = 32, 10 = 128, 11 = 256
sw[12:10], sw[15]	Not used	-

Timing

כמו במעבדה הקודמת, כל דרישה של זמן תורגמה למספר מחזורים של שעון המערכת (100MHz), וממספר המחזורים חושב רוחב המונה, לפי הנוסחה bits = ceil(log2(N)).

Requirement	Count	Counter width
UART bit, 57600 baud	1,736 cycles of 100MHz	11 bits
Button debounce, 15 ms	1,500,000 cycles	21 bits
Delay 50 / 100 / 200 ms	5M / 10M / 20M cycles	25 bits
Digit refresh, 4kHz	25,000 cycles	15 bits

שגיאת הבאוד: 100,000,000 / 57,600 = 1736.11. עיגלתי ל-1736 מחזורים, ולכן קצב השידור בפועל הוא 57,603.7 baud, שגיאה של 0.006% בלבד. מקלט UART דוגם כל ביט באמצעו, ולכן הוא סובל שגיאה של אחוזים בודדים לאורך מסגרת של 10 ביטים. השגיאה שלנו רחוקה מאוד מהגבול.

רענון התצוגה: הפולס של 4kHz קובע את קצב המעבר בין ספרה לספרה. הבקר עובר על 8 ספרות, ולכן כל ספרה נדלקת מחדש 500 פעמים בשנייה, הרבה מעל הקצב שבו העין מבחינה בהבהוב.

Output format

שורה בריבוע מכילה N תווי נתונים ו-N פחות 1 רווחים (0x20) ביניהם, ואחריה CR ו-LF (0x0D 0x0A). אחרי השורה האחרונה לא נשלחים CR ו-LF. כשהגודל הוא 1 נשלח תו בודד, בלי רווח ובלי ירידת שורה. ההשהיה מגיעה רק אחרי תווי נתונים, ולא אחרי רווחים או CR ו-LF. לכן מספר הבתים בריבוע הוא $N^2 + N(N-1) + 2(N-1)$, וזמן השידור הוא מספר הבתים כפול 10 ביטים חלקי 57600, ועוד N^2 השהיות:

N	Data chars	Total bytes	Time, no delay	Time, 50 ms delay
1	1	1	0.17 ms	50 ms
32	1,024	2,078	0.36 s	51.6 s
128	16,384	32,894	5.7 s	13.7 min
256	65,536	131,326	22.8 s	55 min

הטבלה הסבירה גם איך לבחור את התרחישים לבדיקה על הלוח: ריבוע של 32 עם השהיה מאפשר לראות את ההתקדמות בעין, וריבוע של 256 נבדק בלי השהיה.

Display

Digits	Content	Values
an1-an0	Character (hex)	00 - FF
an3-an2	Delay in units of 100 ms, DP on an3	0.0 / 0.5 / 1.0 / 2.0
an5-an4	Size (hex)	01 / 20 / 80 / FF
an7-an6	Rows sent (hex)	counts up N = 256 ends at 00

עד הלחיצה הראשונה אחרי ריסט התצוגה כבויה לגמרי. את גודל 256 מציגים כ-FF, כי בשתי ספרות הקס אי אפשר להציג 100. מונה השורות הוא של 8 ביטים, ולכן בסוף ריבוע של 256 הוא עוטף ומציג 00.

External inputs

- **Metastability:** הכפתור משתנה בלי קשר לשעון, הוא עובר דרך שני פליפ-פלופים בשרשרת לפני שהלוגיקה משתמשת בו.
- **Bouncing:** ה-Debouncer דורש שהכפתור יהיה לחוץ ברצף 15 ms לפני שהוא מוציא פולס. כל קפיצה מאפסת את המונה, ומקבלים פולס אחד בלבד לכל לחיצה.
- **המתגים:** המתגים נדגמים רק ברגע הנעילה, אחרי לפחות 15 ms של לחיצה יציבה. עד אז הם יציבים כבר זמן רב, ולכן לא הוספתי להם סנכרון.
- **Reset:** במעבדה 3 ציינתי כמגבלה ידועה שהריסט לא עובר סנכרון. במעבדה הזו הוספתי reset synchronizer ברמה העליונה: הריסט נכנס מיד (אסינכרונית), אבל משתחרר רק בעליית שעון, אחרי שני פליפ-פלופים. כל הרגיסטרים בתכן יוצאים מהריסט באותו מחזור.

Step 2 - Block Diagram

חילקתי את המערכת לשמונה מודולים, ועוד reset synchronizer קטן ברמה העליונה:

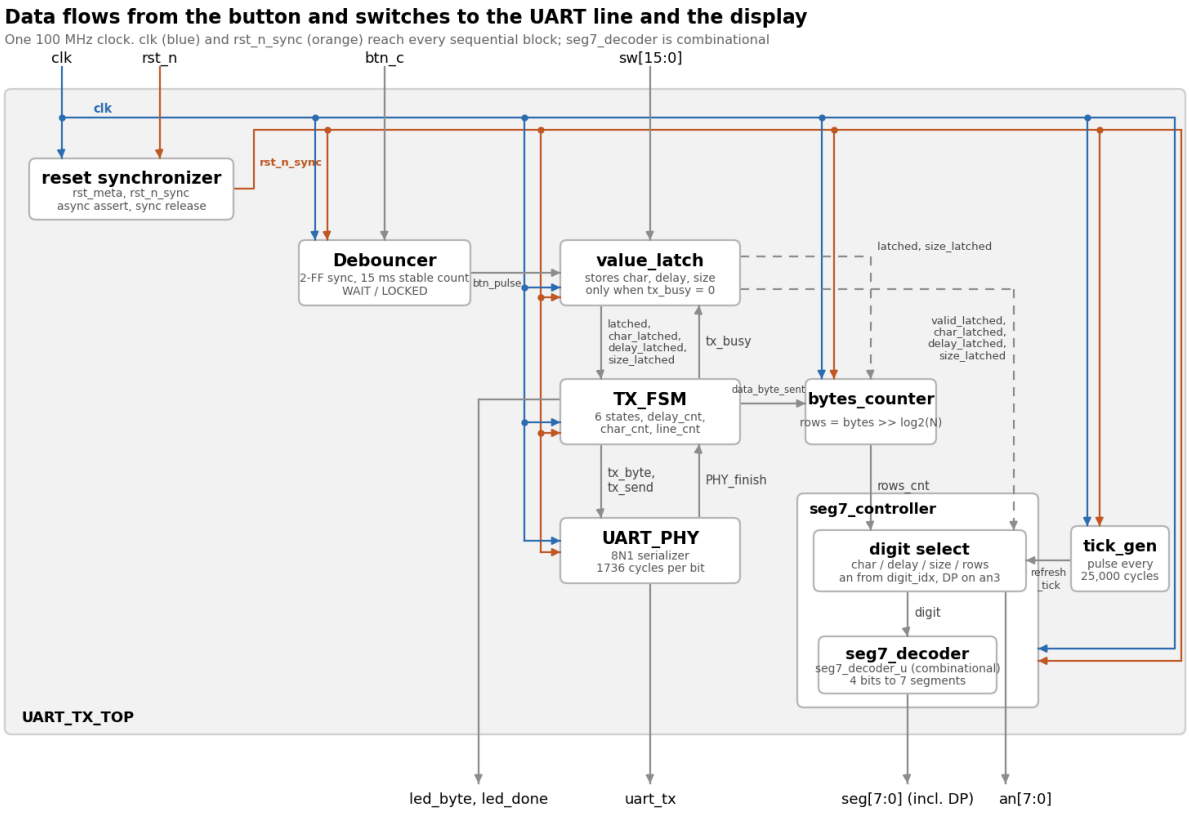


Figure 1 - UART_TX_TOP block diagram

Module	Type	Role
Debouncer	Sequential	סנכרון הכפתור, ספירת 15 ms יציבים ופולס אחד לכל לחיצה
value_latch	Sequential	נעילת התו, ההשהיה והגודל, רק כשהמשדר פנוי
TX_FSM	Sequential	מכונת המצבים הראשית: תווים, רווחים, CR ו-LF והשהיות
UART_PHY	Sequential	שידור בית בפורמט 8N1, ביט אחרי ביט
bytes_counter	Sequential	ספירת תווי הנתונים והמרה למספר שורות לתצוגה
tick_gen	Sequential	פולס של מחזור אחד כל 25,000 מחזורים (4kHz)
seg7_controller	Sequential	סריקת 8 הספרות, בחירת הספרה והנקודה העשרונית
seg7_decoder	Combinational	תרגום ספרת הקס לשבעה סגמנטים, מופע בתוך seg7_controller

Step 3 - Module Design

כל מכונות המצבים בתכן כתובות באותו סגנון של שלושה בלוקים: רגיסטר מצב, בלוק של יציאות רשומות (registered) שמחושבות לפי המצב הבא יחד עם המונים, ובלוק קומבינטורי של המצב הבא. היציאות רשומות, ולכן אין בהן glitches, והן מתעדכנות באותו מחזור שבו המכונה נכנסת למצב.

seg7_decoder

הוא בלוק קומבינטורי שמתרגם ספרה של 4 ביט לתבנית של 7 קטעים, active low. האותיות B ו-D מוצגות כאותיות קטנות, כדי שלא ייראו כמו 8 ו-0. ערך ברירת מחדל בתחילת הבלוק וענף default מונעים latch.

seg7_decoder.v

```
1  `timescale 1ns / 1ps
2
3  module seg7_decoder(
4
5      input wire [3:0] digit,
6      output reg [6:0] seg
7      );
8
9      always @(*) begin
10         seg = 7'b1111111;   // default: all segments off
11         case (digit)
12             4'd0: seg = 7'b1000000;
13             4'd1: seg = 7'b1111001;
14             4'd2: seg = 7'b0100100;
15             4'd3: seg = 7'b0110000;
16             4'd4: seg = 7'b0011001;
17             4'd5: seg = 7'b0010010;
18             4'd6: seg = 7'b0000010;
19             4'd7: seg = 7'b1111000;
20             4'd8: seg = 7'b0000000;
21             4'd9: seg = 7'b0011000;
22             4'hA: seg = 7'b0001000;
23             4'hB: seg = 7'b0000011;
24             4'hC: seg = 7'b1000110;
25             4'hD: seg = 7'b0100001;
26             4'hE: seg = 7'b0000110;
27             4'hF: seg = 7'b0001110;
28             default: seg = 7'b1111111;
29         endcase
30     end
31 endmodule
```

tick_gen

אותו מבנה כמו במעבדה 3, עם פרמטר אחר: מונה של 15 ביט שסופר מ-0 עד 24,999 וחוזר ל-0. בכל פעם שהמונה מגיע ל-MAX, היציאה refresh_tick עולה ל-'1' למשך מחזור אחד, כלומר 4,000 פעמים בשנייה. הפולס משמש כ-enable ולא כשעון.

tick_gen.v

```
1  `timescale 1ns / 1ps
```

```
2
3  module tick_gen #(
4      parameter MAX = 24999
5  )(
6
7      input wire clk, rst_n,
8      output wire refresh_tick
9      );
10
11     reg [14:0] tick_cnt;   // 15 bits cover MAX = 24999
12
13     always @(posedge clk or negedge rst_n) begin
14         if(!rst_n)
15             tick_cnt <= 15'd0;
16         else if (tick_cnt == MAX)
17             tick_cnt <= 15'd0;
18         else
19             tick_cnt <= tick_cnt + 15'd1;
20     end
21
22     assign refresh_tick = (tick_cnt == MAX);   // one-cycle enable pulse
23
24 endmodule
```

Debouncer

הבלוק מורכב משלושה חלקים: סנכרון של שני פליפ-פלופים, מונה stable_cnt של 21 ביט שעולה כל עוד הכפתור לחוץ, נעצר (רווי) ב-BTN_CYCLES ומתאפס ברגע שהכפתור משתחרר, ו-FSM של שני מצבים שמבטיח פולס אחד לכל לחיצה:

From	To	Condition	Output
WAIT	LOCKED	stable_flag (cnt == BTN_CYCLES)	pulse on btn_pulse
LOCKED	WAIT	!any_btn (button released)	-

זה אותו רעיון כמו ב-btn_hold_detector של מעבדה 3, עם שני שינויים: הזמן קצר יותר (15 ms במקום שנייה), והמונה סופר מחזורי שעון ישירות ונעצר ב-BTN_CYCLES, כך שהוא לא עוטף גם בהחזקה ארוכה.

Debouncer.v

```
`timescale 1ns / 1ps

module Debouncer #(
    parameter BTN_CYCLES = 1500000
)(
    input wire clk, rst_n,
    input wire btn_c,
    output wire btn_pulse
    );

    // 2-FF synchronizer for the asynchronous button input
    reg btn_c_meta, btn_c_sync;

    // Button state
    wire any_btn;

    // Counts consecutive cycles with the button held (saturates at BTN_CYCLES)
    reg  [20:0]stable_cnt;   // 21 bits cover 1,500,000
    wire       stable_flag;

    // FSM: WAIT = ready for a new press, LOCKED = press reported, wait for release
    localparam WAIT   = 1'b0;
    localparam LOCKED = 1'b1;
    reg state;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            btn_c_meta <= 1'd0;
            btn_c_sync <= 1'd0;
        end else begin
            btn_c_meta <= btn_c;
            btn_c_sync <= btn_c_meta;
        end
    end

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            state <= WAIT;
        end else begin
            case (state)
                WAIT: begin
                    if (stable_flag) begin
                        state <= LOCKED;
                    end else begin
                        state <= WAIT;
                    end
                end

                LOCKED: begin
                    if (!any_btn) begin
                        state <= WAIT;
                    end else begin
                        state <= LOCKED;
                    end
                end

                default: state <= WAIT;

            endcase
        end
```

```
    end

    always @(posedge clk or negedge rst_n) begin
      if (!rst_n) begin
          stable_cnt <= 21'd0;
      end else if (!any_btn) begin
          stable_cnt <= 21'd0;
      end else if (stable_cnt < BTN_CYCLES) begin
          stable_cnt <= stable_cnt + 21'd1;
      end
    end

    assign any_btn = btn_c_sync;
    assign stable_flag = (stable_cnt == BTN_CYCLES);
    assign btn_pulse = (state == WAIT) && stable_flag;   // one-cycle pulse: FSM moves to LOCKED on the next edge

endmodule
```

value_latch

הבלוק שומר את שלושת השדות מהמתגים. הוא מתעדכן רק כשמגיע btn_pulse וגם המשדר פנוי (tx_busy = 0), ולכן לחיצה באמצע שידור לא משנה כלום. באותו מחזור הוא מוציא פולס latched של מחזור אחד, שמתחיל את השידור, ומדליק את valid_latched, שנשאר דלוק עד הריסט הבא ומאפשר את התצוגה.

value_latch.v

```
`timescale 1ns / 1ps

module value_latch(

    input wire clk, rst_n,
    input wire btn_pulse,
    input wire tx_busy,
    input wire [15:0] sw,
    output reg [7:0] char_latched,
    output reg [1:0] delay_latched,
    output reg [1:0] size_latched,
    output reg latched,
    output reg valid_latched
);

    always @(posedge clk or negedge rst_n) begin
      if (!rst_n) begin
          latched <= 1'b0;
          valid_latched <= 1'b0;
          char_latched <= 8'd0;
          delay_latched <= 2'd0;
          size_latched <= 2'd0;
      // Capture a new configuration only when idle
      end else if (btn_pulse && !tx_busy) begin
          latched <= 1'b1;
          valid_latched <= 1'b1;   // stays high after the first press
          char_latched <= sw[7:0];   // ASCII character
          delay_latched <= sw[9:8];   // 00=0, 01=50, 10=100, 11=200 ms
          size_latched <= sw[14:13];   // 00=1, 01=32, 10=128, 11=256
      end else
          latched <= 1'b0;   // one-cycle pulse; settings are held
    end
endmodule
```

UART_PHY

המשדר הפיזי שולח בית אחד בפורמט 1-8-1: ביט התחלה (0), שמונה ביטי נתונים מה-LSB, וביט עצירה (1). המונה cycle_cnt סופר 1736 מחזורים לכל ביט, והמונה index_cnt סופר את הביטים במסגרת:

index_cnt	Bit	uart_tx
0	Start	0
1 - 8	Data, LSB first	shift_reg[0]
9	Stop	1
10 (IDLE)	Line idle	1

בתחילת מסגרת הבית נטען ל-shift_reg, ואחרי כל ביט נתונים הרגיסטר זז ימינה, כך שהביט הבא תמיד נמצא ב-shift_reg[0]. בסוף ביט העצירה PHY_finish עולה למחזור אחד, וזה הסימן ל-TX_FSM שאפשר לשלוח את הבית הבא. בקשת tx_send שמגיעה באמצע מסגרת מתעלמת.

שתי הערות על התזמון: אחרי מסגרת index_cnt נשאר על 10, ולכן ביט ההתחלה מתחיל מחזור אחד אחרי tx_send. אחרי ריסט index_cnt הוא 0, ולכן ביט ההתחלה של המסגרת הראשונה ארוך במחזור אחד. מחזור אחד מתוך 1736 הוא 0.06% מזמן הביט, וזה לא משפיע על המקלט.

UART_PHY.v

```
1  `timescale 1ns / 1ps
2
3  module UART_PHY #(
4      parameter BIT_CYCLES = 1736
5  )(
6      input wire clk, rst_n,
7      input wire [7:0] tx_byte,
8      input wire tx_send,
9      output reg uart_tx,
10     output reg PHY_finish
11     );
12
13     reg [7:0] shift_reg;
14     reg [10:0] cycle_cnt;   // counts BIT_CYCLES per bit (11 bits cover 1736)
15     reg [3:0] index_cnt;   // 0=start, 1-8=data, 9=stop (reaches 10 back in IDLE)
16     reg state, next;
17
18     localparam IDLE = 1'b0;
19     localparam SENDING = 1'b1;
20
21     always @(posedge clk or negedge rst_n) begin
22         if (!rst_n) begin
23             state <= IDLE;
24         end else
25             state <= next;
26     end
27
28     always @(posedge clk or negedge rst_n) begin
29         if (!rst_n) begin
30             uart_tx <= 1'b1;
31             PHY_finish <= 1'b0;
32             shift_reg <= 8'b0;
33             cycle_cnt <= 11'b0;
34             index_cnt <= 4'b0;
35         end else begin
36             // One-cycle pulse at the end of the stop bit
37             PHY_finish <= (state == SENDING) && (next == IDLE);
38             // Registered line output, driven by the next state and the bit index.
39             // Note: index_cnt is 10 after a frame, so the start bit begins one cycle
40             // after tx_send; after reset index_cnt is 0, so the first start bit is one cycle longer.
41             case (next)
```

```
            IDLE: uart_tx <= 1'b1;
            SENDING: begin
                case (index_cnt)
                4'd0: uart_tx <= 1'b0;
                4'd1, 4'd2, 4'd3, 4'd4, 4'd5, 4'd6, 4'd7, 4'd8: uart_tx <= shift_reg[0];   // data bits, LSB first
                4'd9: uart_tx <= 1'b1;
                default: uart_tx <= 1'b1;
                endcase
            end
            default: uart_tx <= 1'b1;
        endcase

        // Datapath: load byte on tx_send, then count cycles per bit and bits per frame
        case (state)
        IDLE: begin
            if(tx_send) begin
                shift_reg <= tx_byte;
                cycle_cnt <= 11'b0;
                index_cnt <= 4'b0;
            end
        end
        SENDING: begin
            if (cycle_cnt == BIT_CYCLES -1) begin
                cycle_cnt <= 11'b0;
                index_cnt <= index_cnt + 4'd1;
                if (index_cnt >= 4'd1 && index_cnt <= 4'd8)   // shift only after each data bit
                    shift_reg <= shift_reg >> 1;
            end else begin
                cycle_cnt <= cycle_cnt + 11'd1;
            end
        end
        endcase
      end
    end

    // Next-state logic
    always @(*) begin
      case (state)
        IDLE: next = tx_send ? SENDING : IDLE;
        SENDING: next = (index_cnt == 4'd9 && cycle_cnt == BIT_CYCLES - 1) ? IDLE : SENDING;
        default: next = IDLE;
      endcase
    end
endmodule
```

TX_FSM

מה המכונה צריכה לעשות: לשלוח תו, לחכות את ההשהיה, ואז להחליט אם לשלוח רווח, ירידת שורה, או לסיים.
מתוך זה יצאו שישה מצבים:

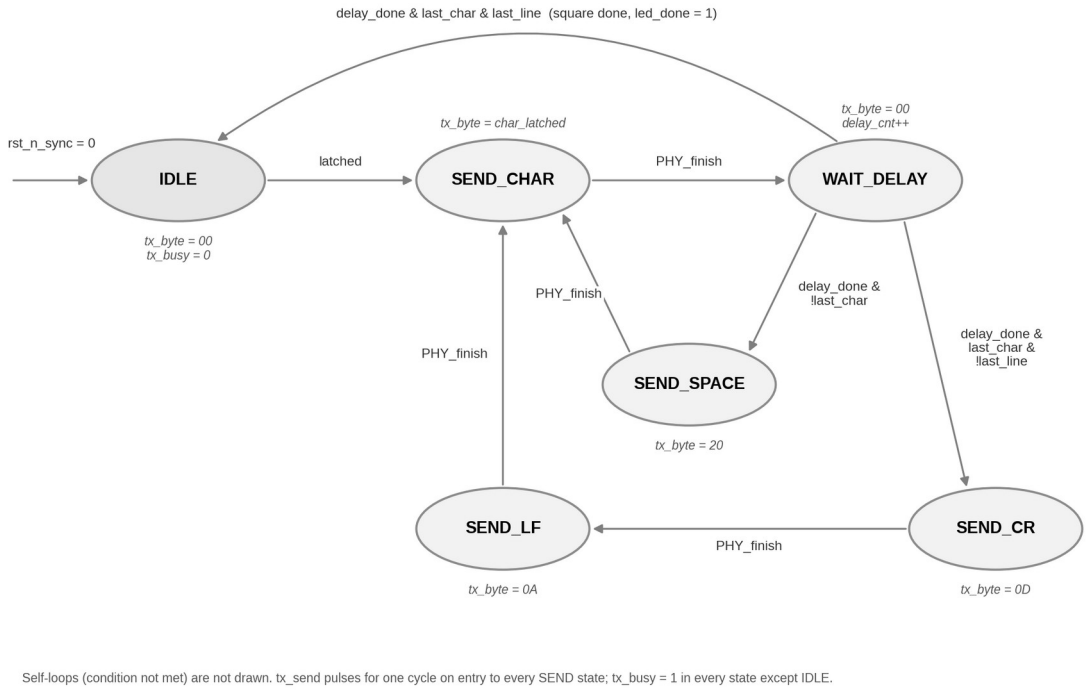


Figure 2 - TX_FSM state diagram

From	To	Condition	tx_byte
IDLE	SEND_CHAR	latched	char_latched
SEND_CHAR	WAIT_DELAY	PHY_finish	00
WAIT_DELAY	SEND_SPACE	delay_done & !last_char	20
WAIT_DELAY	SEND_CR	delay_done & last_char & !last_line	0D
WAIT_DELAY	IDLE	delay_done & last_char & last_line	00
SEND_SPACE	SEND_CHAR	PHY_finish	char_latched
SEND_CR	SEND_LF	PHY_finish	0A
SEND_LF	SEND_CHAR	PHY_finish	char_latched

המונים שנדרשים מתוך המעברים:

Counter	Width	Counts	Cleared
delay_cnt	25	clock cycles in WAIT_DELAY (up to 20,000,000)	outside WAIT_DELAY
char_cnt	9	data chars in the current row (up to 256)	after LF and in IDLE
line_cnt	8	completed rows (up to 255)	in IDLE

מהמונים יוצאים שלושה דגלים: delay_done (ההשהיה הסתיימה, או שהיא 0), last_char (נשלחו N תווים בשורה) ו-last_line (זו השורה האחרונה). המכונה עוברת לשלב הבא רק ב-WAIT_DELAY, אחרי שגם ההשהיה של התו האחרון הסתיימה, ולכן led_done נדלק רק אחרי ההשהיה האחרונה.

החלטת תכן: ל-TX_FSM יש מונים משלו, למרות ש-bytes_counter כבר סופר את תווי הנתונים. בחרתי בזה כדי להפריד בין הבקרה לבין התצוגה. bytes_counter סופר רק תווי נתונים ובשביל התצוגה בלבד, ואילו המכונה צריכה לדעת את המיקום בתוך השורה ואת מספר השורות. שימוש במונה אחד לשני התפקידים היה קושר את הלוגיקה של הבקרה לנתיב של התצוגה.

tx_send עולה למחזור אחד בכניסה לכל אחד ממצבי השליחה, רק כשהמצב הבא שונה מהנוכחי. tx_byte נקבע באותו מחזור לפי המצב הבא, ולכן הבית כבר יציב כשהפיזי דוגם אותו.

```
1  `timescale 1ns / 1ps
2
3  module TX_FSM #(
4
5      parameter DELAY_50MS = 5000000,
6      parameter DELAY_100MS = 10000000,
7      parameter DELAY_200MS = 20000000
8  )(
9      input wire clk, rst_n,
10     input wire [7:0] char_latched,
11     input wire [1:0] delay_latched,
12     input wire [1:0] size_latched,
13     input wire latched,
14     input wire PHY_finish,
15     output reg [7:0] tx_byte,
16     output reg tx_send,
17     output reg tx_busy,
18     output reg data_byte_sent,
19     output reg led_byte,
20     output reg led_done
21 );
22
23     // State encoding (Vivado re-encodes it to one-hot during synthesis)
24     localparam [2:0] IDLE = 3'b000;
25     localparam [2:0] SEND_CHAR = 3'b001;
26     localparam [2:0] WAIT_DELAY = 3'b010;
27     localparam [2:0] SEND_SPACE= 3'b011;
28     localparam [2:0] SEND_CR = 3'b100;
29     localparam [2:0] SEND_LF = 3'b101;
30
31     reg [2:0] state, next;
32     reg [24:0] delay_cnt;   // 25 bits cover 20,000,000 (200 ms)
33     reg [8:0] char_cnt;   // data chars sent in the current row (up to 256)
34     reg [7:0] line_cnt;   // completed rows (CR+LF sent)
35     wire [8:0] row_height;
36     wire [24:0] delay_target;
37     wire delay_done;
38     wire last_char;
39     wire last_line;
40
41     // Row length (= number of rows) according to the latched size
42     assign row_height = (size_latched == 2'd0) ? 9'd1 :
43                         (size_latched == 2'd1) ? 9'd32 :
44                         (size_latched == 2'd2) ? 9'd128 : 9'd256;
45
46     // Delay length in clock cycles according to the latched delay
47     assign delay_target = (delay_latched == 2'd0) ? 25'd0 :
48                           (delay_latched == 2'd1) ? DELAY_50MS :
49                           (delay_latched == 2'd2) ? DELAY_100MS : DELAY_200MS;
50
51     assign delay_done = (delay_target == 25'b0) || (delay_cnt == delay_target - 1);   // zero delay passes immediately
52     assign last_char = (char_cnt == row_height);   // all data chars of the row were sent
53     assign last_line = (line_cnt == row_height - 1);   // current row is the last one
54
55     always @(posedge clk or negedge rst_n) begin
56         if (!rst_n) begin
57             state <= IDLE;
58         end else
59             state <= next;
60     end
61
62     // Registered outputs (decoded from next state) and datapath counters
63     always @(posedge clk or negedge rst_n) begin
```

```
64        if (!rst_n) begin
65            tx_byte <= 8'b0;
66            tx_send <= 1'b0;
67            tx_busy <= 1'b0;
68            data_byte_sent <= 1'b0;
69            delay_cnt <= 25'b0;
70            char_cnt <= 9'b0;
71            line_cnt <= 8'b0;
72            led_byte <= 1'b0;
73            led_done <= 1'b0;
74        end else begin
75            case (next)
76                IDLE: tx_byte <= 8'h0;
77                WAIT_DELAY: tx_byte <= 8'h0;
78                SEND_CHAR: tx_byte <= char_latched;
79                SEND_SPACE: tx_byte <= 8'h20;
80                SEND_CR: tx_byte <= 8'h0D;
81                SEND_LF: tx_byte <= 8'h0A;
82                default: tx_byte <= 8'h00;
83                endcase
84
85                tx_busy <= (next != IDLE);   // high from start until the last delay ends
86                // One-cycle send request on entry to any SEND state
87                tx_send <= ((next == SEND_CHAR) || (next == SEND_SPACE) || (next == SEND_CR) || (next == SEND_LF)) && (next !=
state);
88                data_byte_sent <= (state == SEND_CHAR) && (PHY_finish);   // data chars only (not spaces / CR / LF)
89
90                // led_byte toggles per data char, cleared on a new start
91                if (latched) begin
92                    led_byte <= 1'b0;
93                end else if ((state == SEND_CHAR) && (PHY_finish)) begin
94                    led_byte <= ~led_byte;
95                end
96
97                // led_done: on at completion, off at the next start
98                if ((state == WAIT_DELAY) && (next == IDLE)) begin
99                    led_done <=1'b1;
100               end else if ((state == IDLE) && (next == SEND_CHAR)) begin
101                   led_done <= 1'b0;
102               end
103           // Row / line counters
104           case (state)
105               IDLE: begin
106               char_cnt <= 9'b0;
107               line_cnt <= 8'b0;
108               end
109               SEND_CHAR: if (PHY_finish) begin
110                   char_cnt <= char_cnt + 9'd1;
111               end
112               SEND_LF: if (PHY_finish) begin
113                   char_cnt <= 9'b0;
114                   line_cnt <= line_cnt + 8'd1;
115               end
116               default: ;
117           endcase
118           // Delay counter runs only in WAIT_DELAY
119           delay_cnt <= (state == WAIT_DELAY) ? delay_cnt + 25'd1 : 25'b0;
120           end
121   end
122
123   // Next-state logic
124   always @(*) begin
125       case (state)
126           IDLE: next = (latched) ? SEND_CHAR : IDLE;
127           SEND_CHAR: next = (PHY_finish) ? WAIT_DELAY : SEND_CHAR;
```

```
128         WAIT_DELAY: next = (!delay_done) ? WAIT_DELAY :   // after each data char: delay, then space / CR+LF / done
129                     (!last_char) ? SEND_SPACE :
130                     (!last_line) ? SEND_CR : IDLE;
131         SEND_SPACE: next = (PHY_finish) ? SEND_CHAR : SEND_SPACE;
132         SEND_CR: next = (PHY_finish) ? SEND_LF : SEND_CR;
133         SEND_LF: next = (PHY_finish) ? SEND_CHAR : SEND_LF;
134         default: next = IDLE;
135       endcase
136     end
137 endmodule
```

bytes_counter

מונה של 17 ביט שעולה בכל data_byte_sent ומתאפס בכל התחלה חדשה. 17 ביט נדרשים כדי לספור עד 65,536 תווים. מספר השורות מתקבל בחלוקה ב-N, וכיוון ש-N הוא חזקה של 2, החלוקה היא הזזה ימינה ב-log2(N): 0, 5, 7 או 8. לתצוגה נכנסים רק 8 הביטים התחתונים של התוצאה, ולכן בגודל 256 המונה עוטף ל-00, כמו שהוחלט בשלב הדרישות.

bytes_counter.v

```
1  `timescale 1ns / 1ps
2
3  module bytes_counter(
4
5     input wire clk, rst_n,
6     input wire data_byte_sent,
7     input wire latched,
8     input wire [1:0] size_latched,
9     output wire [7:0] rows_cnt
10     );
11
12     reg [16:0] byte_cnt;   // up to 65,536 data chars (256x256)
13     wire [3:0] shift_amt;
14     wire [16:0] rows_full;
15
16     // Shift amount = log2(N)
17     assign shift_amt =  (size_latched == 2'd0) ? 4'd0 :
18                         (size_latched == 2'd1) ? 4'd5 :
19                         (size_latched == 2'd2) ? 4'd7 : 4'd8;
20
21     assign rows_full = byte_cnt >> shift_amt;
22     assign rows_cnt = rows_full[7:0];   // 8 bits displayed; N=256 wraps to 00 by spec (lint waiver ASSIGN-6)
23
24     always @(posedge clk or negedge rst_n) begin
25        if (!rst_n) begin
26           byte_cnt <= 17'b0;
27        end else if (latched) begin   // clear on a new start
28           byte_cnt <= 17'b0;
29        end else if (data_byte_sent) begin
30           byte_cnt <= byte_cnt + 17'd1;
31        end
32     end
33  endmodule
```

seg7_controller

מונה digit_idx של 3 ביט מתקדם בכל refresh_tick, מ-0 עד 7 ובחזרה ל-0. לפי הערך שלו נבחרת הספרה הנוכחית, נקבעת האנודה (active low), והנקודה העשרונית נדלקת רק בספרה an3, כך שההשהיה מוצגת כ-0.5,

1.0 וכדומה. כל עוד valid_latched = 0, כל האנודות על '1' והתצוגה כבויה. הערכים של ההשהיה והגודל מחושבים מהקוד הבינארי בעזרת שרשרת ternary, בלי צורך בהמרה ל-BCD.

seg7_controller.v

```
`timescale 1ns / 1ps

module seg7_controller(

   input wire clk, rst_n,
   input wire refresh_tick,
   input wire [7:0] char_latched,
   input wire [1:0] delay_latched,
   input wire [1:0] size_latched,
    input wire [7:0] rows_cnt,
    input wire valid_latched,
    output wire [7:0] an,
    output wire [7:0] seg
    );

    reg [2:0] digit_idx;   // active digit, advanced by refresh_tick
    wire [3:0] digit;
    wire [3:0] delay_hi, delay_low, size_hi, size_low;
    wire dp;
    wire [6:0] seg_raw;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            digit_idx <= 3'd0;
        end else if (refresh_tick) begin
                digit_idx <= digit_idx + 3'd1;
        end
    end

    // Active-low anodes; all digits off until the first press
    assign an = (valid_latched) ? ~(8'd1 << digit_idx) : 8'hFF;

    // Delay digits in units of 100 ms: 0.0 / 0.5 / 1.0 / 2.0
    assign delay_hi = (delay_latched == 2'd0) ? 4'd0 :
                      (delay_latched == 2'd1) ? 4'd0 :
                      (delay_latched == 2'd2) ? 4'd1 : 4'd2;

    assign delay_low = (delay_latched == 2'd0) ? 4'd0 :
                       (delay_latched == 2'd1) ? 4'd5 :
                       (delay_latched == 2'd2) ? 4'd0 : 4'd0;

    // Size digits in hex: 01 / 20 / 80 / FF
    assign size_hi = (size_latched == 2'd0) ? 4'd0 :
                     (size_latched == 2'd1) ? 4'd2 :
                     (size_latched == 2'd2) ? 4'd8 : 4'hF;

    assign size_low = (size_latched == 2'd0) ? 4'd1 :
                      (size_latched == 2'd1) ? 4'd0 :
                      (size_latched == 2'd2) ? 4'd0 : 4'hF;

    // Digit select
    assign digit = (digit_idx == 3'd0) ? char_latched [3:0] :
                   (digit_idx == 3'd1) ? char_latched [7:4] :
                   (digit_idx == 3'd2) ? delay_low :
                   (digit_idx == 3'd3) ? delay_hi :
                   (digit_idx == 3'd4) ? size_low :
                   (digit_idx == 3'd5) ? size_hi :
                   (digit_idx == 3'd6) ? rows_cnt [3:0] : rows_cnt [7:4];

     assign dp = (digit_idx == 3) ? 1'b0 : 1'b1;   // decimal point (active low) on an3
```

```
61
62      seg7_decoder seg7_decoder_u (
63          .digit(digit),
64          .seg(seg_raw)
65      );
66
67      assign seg = {dp, seg_raw};   // seg[7]=DP, seg[6:0]=CG..CA
68
69  endmodule
```

UART_TX_TOP

המודול העליון מחבר את כל הבלוקים, ומכיל את ה-reset synchronizer. כל הפרמטרים של הזמנים (BTN_CYCLES, BIT_CYCLES, DELAYS, REFRESH_MAX) מועברים מלמעלה. ערכי ברירת המחדל הם הערכים האמיתיים לחומרה, ורק הטסטבנצ'ים מחליפים אותם בערכים קטנים, כך שלא צריך לשנות קוד לפני הצריבה.

UART_TX_TOP.v

```
1   `timescale 1ns / 1ps
2
3   module UART_TX_TOP #(
4
5       parameter BTN_CYCLES = 1500000,   // 15 ms debounce @ 100 MHz
6       parameter BIT_CYCLES = 1736,   // 100 MHz / 57600 baud
7       parameter DELAY_50MS = 5000000,   // 50 ms @ 100 MHz
8       parameter DELAY_100MS = 10000000,   // 100 ms @ 100 MHz
9       parameter DELAY_200MS = 20000000,   // 200 ms @ 100 MHz
10      parameter REFRESH_MAX = 24999   // 4 kHz digit refresh (500 Hz full display cycle)
11  )(
12      input wire clk, rst_n,   // rst_n = CPU_RESETN (active low)
13      input wire btn_c,
14      input wire [15:0] sw,   // sw[7:0]=char, sw[9:8]=delay, sw[14:13]=size
15      output wire uart_tx,
16      output wire led_byte, led_done,
17      output wire [7:0] an,
18      output wire [7:0] seg
19  );
20
21      wire btn_pulse;
22      wire [7:0] char_latched;
23      wire [1:0] delay_latched;
24      wire [1:0] size_latched;
25      wire latched, valid_latched;
26      wire tx_busy;
27      wire [7:0] tx_byte;
28      wire tx_send, PHY_finish, data_byte_sent;
29      wire [7:0] rows_cnt;
30      wire refresh_tick;
31
32      // Reset synchronizer: asynchronous assert, synchronous de-assert
33      reg rst_n_sync;
34      reg rst_meta;
35
36      always @(posedge clk or negedge rst_n) begin
37          if(!rst_n) begin
38          rst_n_sync <= 1'b0;
39          rst_meta <= 1'b0;
40          end else begin
41              rst_meta <= 1'b1;
42              rst_n_sync <= rst_meta;
43          end
44      end
```

```
45
46      // Button debounce -> single-cycle press pulse
47      Debouncer#(.BTN_CYCLES(BTN_CYCLES)) Debouncer_u (
48          .clk(clk),
49          .rst_n(rst_n_sync),
50          .btn_c(btn_c),
51          .btn_pulse(btn_pulse)
52      );
53
54      // Latch switches on a press (ignored while transmitting)
55      value_latch value_latch_u(
56          .clk(clk),
57          .rst_n(rst_n_sync),
58          .btn_pulse(btn_pulse),
59          .tx_busy(tx_busy),
60          .sw(sw),
61          .char_latched(char_latched),
62          .delay_latched(delay_latched),
63          .size_latched(size_latched),
64          .latched(latched),
65          .valid_latched(valid_latched)
66      );
67
68      // Main control FSM: sequences data chars, spaces, CR/LF and delays
69      TX_FSM #(.DELAY_50MS(DELAY_50MS), .DELAY_100MS(DELAY_100MS), .DELAY_200MS(DELAY_200MS)) TX_FSM_u(
70          .clk(clk),
71          .rst_n(rst_n_sync),
72          .char_latched(char_latched),
73          .delay_latched(delay_latched),
74          .size_latched(size_latched),
75          .latched(latched),
76          .tx_send(tx_send),
77          .PHY_finish(PHY_finish),
78          .tx_byte(tx_byte),
79          .tx_busy(tx_busy),
80          .data_byte_sent(data_byte_sent),
81          .led_byte(led_byte),
82          .led_done(led_done)
83      );
84
85      // UART serializer: start bit, 8 data bits LSB first, stop bit
86      UART_PHY #(.BIT_CYCLES(BIT_CYCLES)) UART_PHY_u(
87          .clk(clk),
88          .rst_n(rst_n_sync),
89          .tx_byte(tx_byte),
90          .tx_send(tx_send),
91          .uart_tx(uart_tx),
92          .PHY_finish(PHY_finish)
93      );
94
95      // Counts data chars and converts them to completed rows for the display
96      bytes_counter bytes_counter_u (
97          .clk(clk),
98          .rst_n(rst_n_sync),
99          .data_byte_sent(data_byte_sent),
100         .latched(latched),
101         .size_latched(size_latched),
102         .rows_cnt(rows_cnt)
103     );
104
105     // Digit refresh enable pulse (an enable, not a derived clock - avoids CDC)
106     tick_gen #(.MAX(REFRESH_MAX)) tick_gen_u (
107         .clk(clk),
108         .rst_n(rst_n_sync),
109         .refresh_tick(refresh_tick)
```

```
    );

    // 8-digit multiplexed 7-segment display
    seg7_controller seg7_controller_u (
        .clk(clk),
        .rst_n(rst_n_sync),
        .refresh_tick(refresh_tick),
        .char_latched(char_latched),
        .delay_latched(delay_latched),
        .size_latched(size_latched),
        .rows_cnt(rows_cnt),
        .valid_latched(valid_latched),
        .an(an),
        .seg(seg)
    );
endmodule
```

Step 4 - Testbenches & Simulation Results

לכל בלוק עם לוגיקה משמעותית כתבתי טסטבנץ' נפרד, ובסוף טסטבנץ' שבודק את כל המערכת. בכל הטסטבנצ'ים יחידת הזמן היא timescale 1ns/1ps, והשעון מתחיל ב-0, כך שעליות השעון נופלות באמצע בין הזמנים שבהם הכניסות משתנות. tick_gen ו-seg7_decoder נבדקו במעבדה 3, וכאן הם נבדקים כחלק מהטסטבנצ'ים של התצוגה ושל המערכת.

Debouncer_TB

BTN_CYCLES = 10, לחיצה צריכה להיות יציבה 100ns כדי להיחשב.

Debouncer_TB.v

```
`timescale 1ns / 1ps

module Debouncer_TB;

    reg clk, rst_n;
    reg btn_c;
    wire btn_pulse;

    always #5 clk = ~clk;   // 100 MHz clock

     Debouncer #(.BTN_CYCLES(10)) Debouncer_TB_DUT(   // short debounce for simulation

        .clk(clk),
        .rst_n(rst_n),
        .btn_c(btn_c),
        .btn_pulse(btn_pulse)
    );

    initial begin
        clk = 1'b0;
        rst_n = 1'b0;
        btn_c = 1'b0;

        #20;
        rst_n = 1'b1;
        // Scenario 1: clean press longer than BTN_CYCLES - expect one btn_pulse.
        btn_c = 1'b1; #120;
        btn_c = 1'b0;
        #200;
        // Scenario 2: long hold (360 ns) - expect a single pulse, no repeats while held.
        btn_c = 1'b1; #360;
        btn_c = 1'b0; #20;
```

```
        #200;
        // Scenario 3: short glitch (30 ns) shorter than BTN_CYCLES - expect no pulse.
        btn_c = 1'b1; #30;
        btn_c = 1'b0;
        #200;
        // Scenario 4: bouncing press that settles high - expect one pulse once stable.
        btn_c = 1'b1; #30;
        btn_c = 1'b0; #30;
        btn_c = 1'b1; #30;
        btn_c = 1'b0; #30;
        btn_c = 1'b1; #30;
        btn_c = 1'b0; #30;
        btn_c = 1'b1; #120;
        btn_c = 1'b0;
        #200;
        // Scenario 5: stable press followed by a bouncing release - expect one pulse only.
        btn_c = 1'b1; #120;
        btn_c = 1'b0; #30;
        btn_c = 1'b1; #30;
        btn_c = 1'b0; #30;
        btn_c = 1'b1; #30;
        btn_c = 1'b0;
        #200;
        // Scenario 6: two separate presses - expect two pulses.
        btn_c = 1'b1; #120;
        btn_c = 1'b0; #100;
        btn_c = 1'b1; #120;
        btn_c = 1'b0;
        #200;
        // Scenario 7: reset while held - expect no pulse in reset, then a new pulse since the button is still high.
        btn_c = 1'b1; #80;
        rst_n = 1'b0; #120;
        rst_n = 1'b1; #120;
        #200;
        $finish;
    end

endmodule
```

Scenario	Test	Expected
1	Clean press, 120 ns	One btn_pulse
2	Long hold, 360 ns	One pulse only
3	Glitch, 30 ns	No pulse
4	Bouncing press, then stable	One pulse once stable
5	Stable press, bouncing release	One pulse only
6	Two separate presses	Two pulses
7	Reset while held	No pulse in reset, new pulse after it

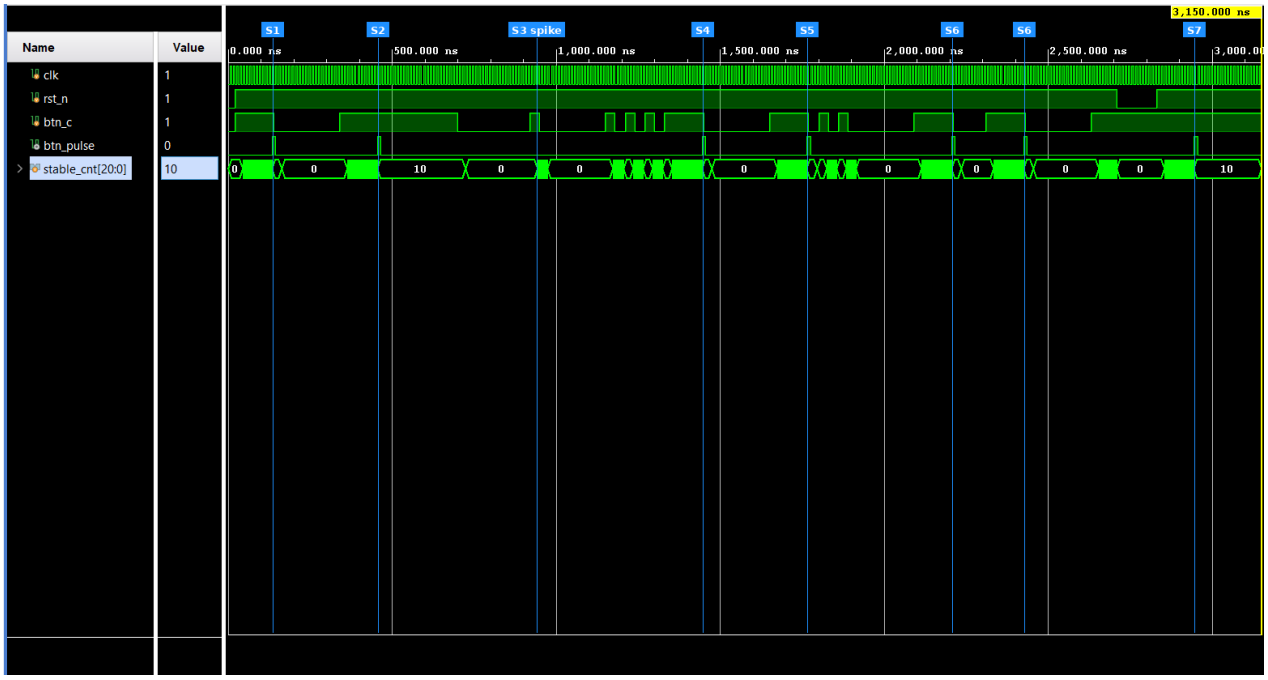


Figure 3 - Debouncer: one pulse per press, glitches and bounces filtered

בסימולציה BTN_CYCLES נדרס ל-10, כך שלחיצה נחשבת יציבה אחרי 100ns. המרקרים מסמנים את שבעת התרחישים. ב-S1 וב-S2 יוצא פולס אחד בלבד, גם כשהכפתור מוחזק זמן רב, כי המכונה עוברת ל-LOCKED. ב-S3 הספייק של 30ns קצר מהסף, stable_cnt מתאפס ואין פולס. ב-S4 כל קפיצה מאפסת את המונה, והפולס יוצא רק אחרי שהכפתור מתייצב. ב-S5 הקפיצות בזמן השחרור לא מוציאות פולס. ב-S6 שתי לחיצות נפרדות נותנות שני פולסים. ב-S7 לחיצה של 80ns לא מספיקה לפני הריסט, ואחרי שחרור הריסט, כשהכפתור עדיין לחוץ, יוצא פולס אחד. בסך הכל 7 פולסים, כמו שחושב מראש. בהחזקה ארוכה, כמו ב-S2 וב-S7, רואים ש-stable_cnt נעצר על 10 ונשאר שם עד השחרור, כך שהמונה לא גולש גם בלחיצה ארוכה מאוד על הלוח.

value_latch_TB

value_latch_TB.v

```
`timescale 1ns / 1ps

module value_latch_TB;

    reg clk, rst_n;
    reg btn_pulse;
    reg tx_busy;
    reg [15:0] sw;
    wire [7:0] char_latched;
     wire [1:0] delay_latched;
     wire [1:0] size_latched;
     wire latched;
     wire valid_latched;

    value_latch value_latch_TB_DUT (

        .clk(clk),
        .rst_n(rst_n),
        .btn_pulse(btn_pulse),
        .tx_busy(tx_busy),
        .sw(sw),
        .char_latched(char_latched),
        .delay_latched(delay_latched),
        .size_latched(size_latched),
        .latched(latched),
        .valid_latched(valid_latched)
     );
```

```
28
29      always #5 clk = ~clk;
30
31      initial begin
32          clk = 1'b0;
33          rst_n = 1'b0;
34          btn_pulse = 1'b0;
35          tx_busy = 1'b0;
36          sw = 16'b0;
37          #100;
38
39          rst_n = 1'b1;
40          #20;
41          // Scenario 1: press while idle - expect char 41, delay code 10, size code 01 and a one-cycle latched pulse.
42          sw = 16'h2241;
43          btn_pulse = 1'b1; #10;
44          btn_pulse = 1'b0;
45          #100;
46
47          // Scenario 2: switches change without a press - expect the latched values to stay unchanged.
48          sw = 16'h6342; #30;
49          #100;
50
51          // Scenario 3: press while tx_busy is high - expect the press to be ignored.
52          tx_busy = 1'b1;
53          btn_pulse = 1'b1; #10;
54          btn_pulse = 1'b0;
55          #100;
56
57          // Scenario 4: press after tx_busy drops - expect char 42, delay code 11, size code 11 to be latched.
58          tx_busy = 1'b0;
59          btn_pulse = 1'b1; #10;
60          btn_pulse = 1'b0;
61          #100;
62
63          // Scenario 5: reset - expect all outputs cleared, including valid_latched.
64          rst_n = 1'b0;
65          #30;
66          rst_n = 1'b1;
67          #100;
68          $finish;
69      end
70  endmodule
```

Scenario	Test	Expected
1	Press while idle, sw = 2241	char 41, delay 10, size 01, one latched pulse
2	Switches change, no press	Latched values unchanged
3	Press while tx_busy = 1	Ignored
4	Press after tx_busy drops, sw = 6342	char 42, delay 11, size 11
5	Reset	All outputs cleared, valid_latched = 0

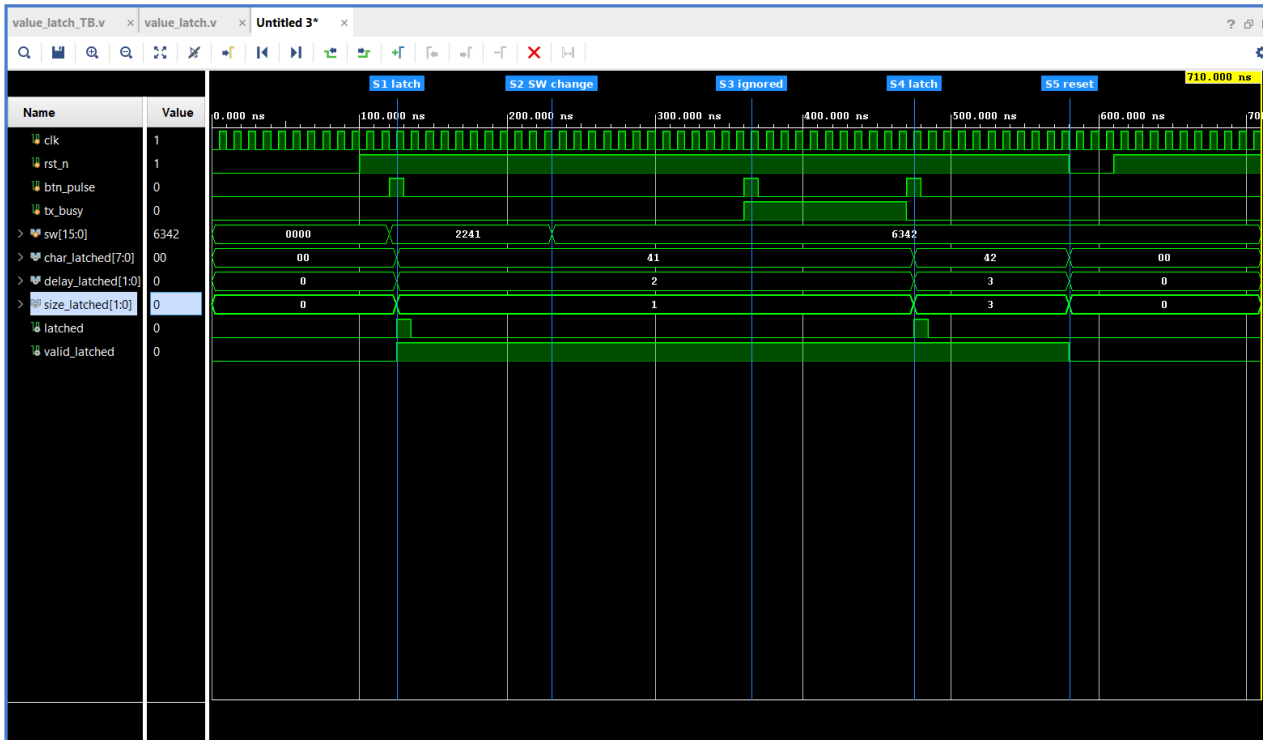


Figure 4 - value_latch: values change only on a press while idle

ב- S1 (125 ns) הלחיצה הראשונה נועלת את הערכים 41, 2 ו-1, ו-latched דולק למחזור אחד באותו קצה שעון שבו הערכים מתעדכנים. valid_latched נדלק ונשאר דולק. ב-S2 (230 ns) המתגים משתנים ל-6342 בלי לחיצה, ושום דבר לא משתנה. ב-S3 (365ns) יש לחיצה בזמן ש-tx_busy דולק, והבלוק מתעלם ממנה: הערכים נשארים ו-latched לא דולק. ב-S4 (475 ns) הלחיצה מגיעה אחרי שהשידור הסתיים, והערכים מתעדכנים ל-42, 3 ו-3. ב-S5 (580ns) הריסט מאפס את כל היציאות, כולל valid_latched.

UART_PHY_TB

BIT_CYCLES = 10, כלומר ביט נמשך 100ns ומסגרת שלמה 1us. ב-0x0F הביטים הם 1111 ואז 0000, כך שקל לראות בגלים שהסדר הוא מה-LSB.

UART_PHY_TB.v

```
1  `timescale 1ns / 1ps
2
3  module UART_PHY_TB;
4
5      reg clk, rst_n;
6      reg [7:0] tx_byte;
7      reg tx_send;
8      wire uart_tx;
9      wire PHY_finish;
10
11     UART_PHY #(.BIT_CYCLES(10)) UART_PHY_TB_DUT(
12
13         .clk(clk),
14         .rst_n(rst_n),
15         .tx_byte(tx_byte),
16         .tx_send(tx_send),
17         .uart_tx(uart_tx),
18         .PHY_finish(PHY_finish)
19     );
20
21     always #5 clk = ~clk;
22
23     initial begin
24
```

```
25        clk = 1'b0;
26        rst_n = 1'b0;
27        tx_send = 1'b0;
28        tx_byte = 8'b0;
29        #20;
30        rst_n = 1'b1;
31        #100;
32
33        // Scenario 1: single frame of 0x0F - expect start bit, data LSB first, stop bit, then PHY_finish.
34        tx_byte = 8'h0F;
35        tx_send = 1'b1; #10;
36        tx_send = 1'b0;
37        #1100;
38
39        // Scenario 2: 0x41 then 0x0F sent right after PHY_finish - expect back-to-back frames with no gap.
40        tx_byte = 8'h41;
41        tx_send = 1'b1; #10;
42        tx_send = 1'b0;
43        @(posedge PHY_finish);
44        @(negedge clk);
45        tx_byte = 8'h0F;
46        tx_send = 1'b1; #10;
47        tx_send = 1'b0;
48        #1100;
49
50        // Scenario 3: tx_send of 0xFF during a frame - expect it ignored and 0x41 to complete normally.
51        tx_byte = 8'h41;
52        tx_send = 1'b1; #10;
53        tx_send = 1'b0;
54        #400;
55        tx_byte = 8'hFF;
56        tx_send = 1'b1; #10;
57        tx_send = 1'b0;
58        #1100;
59
60        // Scenario 4: reset in the middle of a frame - expect the line to return high, then a normal 0x41 frame.
61        tx_byte = 8'h0F;
62        tx_send = 1'b1; #10;
63        tx_send = 1'b0;
64        #400;
65        rst_n = 1'b0; #30;
66        rst_n = 1'b1; #30;
67        tx_byte = 8'h41;
68        tx_send = 1'b1; #10;
69        tx_send = 1'b0;
70        #1100;
71
72        $finish;
73    end
74 endmodule
```

Scenario	Test	Expected
1	Single frame, 0x0F	Start, 1111 0000 (LSB first), stop, PHY_finish
2	0x41, then 0x0F right after PHY_finish	Back-to-back frames, no gap
3	tx_send (0xFF) during a frame	Ignored, 0x41 completes normally
4	Reset mid-frame, then 0x41	Line returns high, new frame is correct



Figure 5 - UART_PHY: all four scenarios

בסימולציה BIT_CYCLES נדרס ל-10, כך שביט נמשך 100 ns ומסגרת שלמה כ-1µs. ב-S1 נשלח הבית 0x0F. ב-S2 נשלחים שני בתים צמודים: 0x41, ומיד אחרי PHY_finish הבית 0x0F, כמו שמכונת המצבים תעשה. ב-S3 מגיע tx_send עם 0xFF באמצע המסגרת של 0x41, וה-PHY מתעלם ממנו, כי הוא בודק את tx_send רק ב-IDLE. ב-S4 ריסט באמצע מסגרת מחזיר את הקו מיד ל-1 בלי PHY_finish, והבית שנשלח אחריו יוצא תקין. PHY_finish עלה 5 פעמים, כמו שחושב מראש.

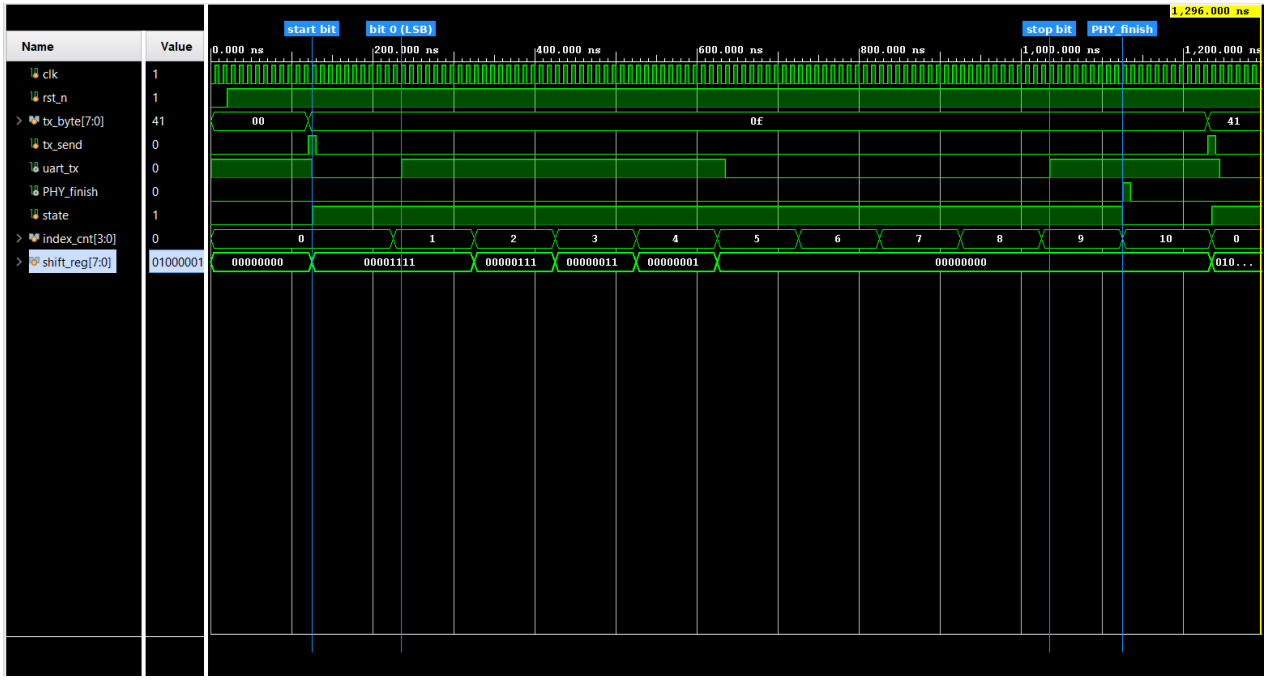


Figure 5b - UART_PHY: 8N1 frame, LSB first, PHY_finish at the end of the stop bit

על הקו רואים ביט התחלה 0, ואחריו ארבעה 1 וארבעה 0, כלומר 0x0F מה-LSB, ובסוף ביט עצירה 1. ב-shift_reg רואים את ההזזה ימינה בסוף כל ביט נתונים, כך שהביט הבא תמיד נמצא ב-.shift_reg[0] PHY_finish עולה למחזור אחד בסוף ביט העצירה. ביט ההתחלה הראשון ארוך במחזור אחד, 110ns, כי uart_tx הוא פלט רשום שמחושב מ-index_cnt של המחזור הקודם. בחומרה זה מחזור אחד מתוך 1,736, וזה זניח.

TX_FSM_TB

במקום המשדר הפיזי, הטסטבנץ' מכיל מודל פשוט שעונה על כל tx_send ב-PHY_finish כמה מחזורים אחר כך. ההשהיות מקוצרות ל-5, 10 ו-20 מחזורים. בתרחיש 3 הטסטבנץ' סופר את פולסי data_byte_sent ומדפיס את המספר בסוף, והתוצאה הייתה 1024, בדיוק 32 כפול 32.

TX_FSM_TB.v

```v
`timescale 1ns / 1ps

module TX_FSM_TB;

    reg clk, rst_n;
    reg [7:0] char_latched;
    reg [1:0] delay_latched;
    reg [1:0] size_latched;
    reg latched;
     reg PHY_finish = 1'b0;
     wire [7:0] tx_byte;
     wire tx_send, tx_busy;
     wire data_byte_sent;
     wire led_byte, led_done;
     reg [31:0] data_cnt;

     TX_FSM #(.DELAY_50MS(5), .DELAY_100MS(10), .DELAY_200MS(20)) TX_FSM_TB_DUT(

         .clk(clk),
         .rst_n(rst_n),
         .char_latched(char_latched),
         .delay_latched(delay_latched),
         .size_latched(size_latched),
         .latched(latched),
         .PHY_finish(PHY_finish),
         .tx_byte(tx_byte),
         .tx_send(tx_send),
         .tx_busy(tx_busy),
         .data_byte_sent(data_byte_sent),
         .led_byte(led_byte),
         .led_done(led_done)
      );

      always #5 clk = ~clk;

      // Count data bytes reported by the FSM
      always @(posedge data_byte_sent)
        data_cnt = data_cnt + 1;

      // Simple PHY model: answers each tx_send with a one-cycle PHY_finish a few cycles later
      integer phy_wait = -1;   // -1 = idle, otherwise cycles left

      always @(negedge clk) begin
          PHY_finish <= 1'b0;               // default: one-cycle pulse
          if (tx_send)
              phy_wait <= 3;                // new request: start counting
          else if (phy_wait > 0)
              phy_wait <= phy_wait - 1;       // still "sending"
          else if (phy_wait == 0) begin
              PHY_finish <= 1'b1;           // done
```

```
51            phy_wait   <= -1;
52          end
53        end
54
55        initial begin
56          clk = 1'b0;
57          rst_n = 1'b0;
58          char_latched = 8'b0;
59          delay_latched = 2'b0;
60          size_latched = 2'b0;
61          latched = 1'b0;
62          data_cnt = 32'b0;
63          #100;
64
65          @(negedge clk);
66          rst_n = 1'b1;
67          @(negedge clk);
68          // Scenario 1: single char 'A' (size 1, no delay) - expect one data byte, no space / CR / LF, led_done set.
69          char_latched = 8'h41;
70          delay_latched = 2'd0;
71          size_latched = 2'd0;
72          latched = 1'b1; #10;
73          latched = 1'b0;
74          @(negedge tx_busy);
75          #100;
76
77          // Scenario 2: second single char 'B' - expect counters restarted, led_byte and led_done cleared then updated.
78          @(negedge clk);
79          char_latched = 8'h42;
80          delay_latched = 2'd0;
81          size_latched = 2'd0;
82          latched = 1'b1; #10;
83          latched = 1'b0;
84          @(negedge tx_busy);
85          #100;
86
87          // Scenario 3: 32x32 square of 'B' with 50 ms delay (5 cycles) - expect 1024 data bytes, spaces and CR/LF.
88          @(negedge clk);
89          data_cnt = 32'b0;
90          @(negedge clk);
91          char_latched = 8'h42;
92          delay_latched = 2'd1;
93          size_latched = 2'd1;
94          latched = 1'b1; #10;
95          latched = 1'b0;
96          @(negedge tx_busy);
97          $display("data bytes send = %0d", data_cnt);
98          #100;
99
100         // Scenario 4: reset during a 32x32 transmission - expect return to IDLE with all outputs cleared.
101         @(negedge clk);
102         char_latched = 8'h42;
103         delay_latched = 2'd1;
104         size_latched = 2'd1;
105         latched = 1'b1; #10;
106         latched = 1'b0; #80000;
107         rst_n = 1'b0; #20;
108         rst_n = 1'b1;
109         #100;
110
111         $finish;
112       end
113   endmodule
```

Scenario	Test	Expected
1	Single char 'A', size 1, no delay	One data byte, no space / CR / LF, led_done = 1
2	Second single char 'B'	Counters restart, LEDs cleared and updated
3	32x32 of 'B', 5-cycle delay	1024 data bytes, spaces, CR+LF between rows
4	Reset during 32x32	Back to IDLE, all outputs cleared

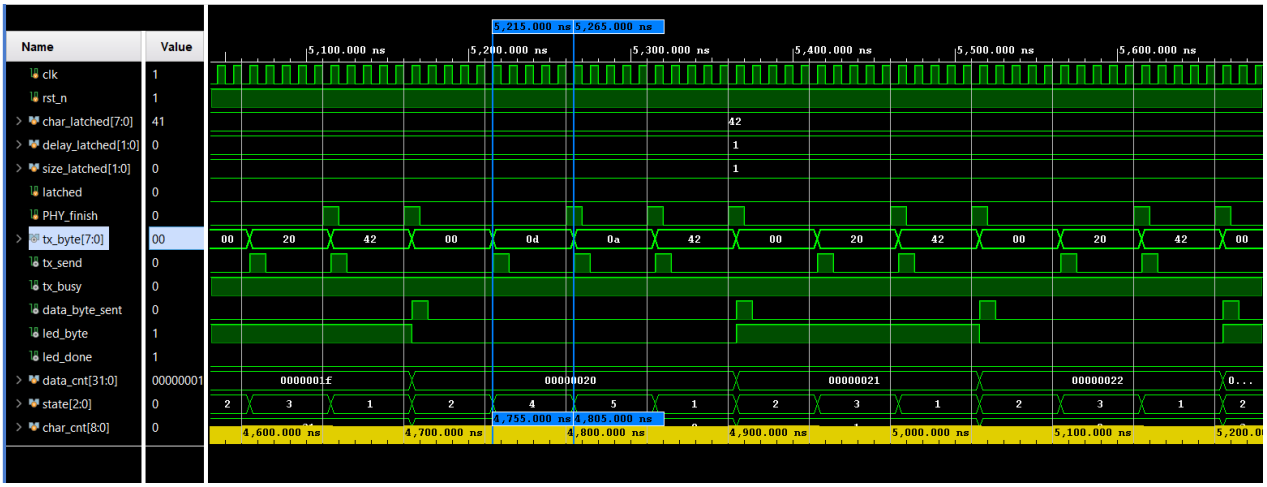


Figure 6 - TX_FSM: end of a row, the delay, CR and LF, then the next row

bytes_counter_TB

bytes_counter_TB.v

```
1  `timescale 1ns / 1ps
2
3  module bytes_counter_TB;
4
5      reg clk, rst_n;
6      reg data_byte_sent;
7      reg latched;
8      reg [1:0] size_latched;
9      wire [7:0] rows_cnt;
10
11     bytes_counter bytes_counter_TB_DUT (
12         .clk(clk),
13         .rst_n(rst_n),
14         .data_byte_sent(data_byte_sent),
15         .latched(latched),
16         .size_latched(size_latched),
17         .rows_cnt(rows_cnt)
18     );
19
20     always #5 clk = ~clk;
21
22     initial begin
23         clk = 1'b0;
24         rst_n = 1'b0;
25         data_byte_sent = 1'b0;
26         latched = 1'b0;
27         size_latched = 2'd0;
28         #20;
29         rst_n = 1'b1;
30         #100;
31
32         // Scenario 1: size 1, one data byte - expect rows_cnt = 01.
33         size_latched = 2'd0;
```

```
        latched = 1'b1; #10;
        latched = 1'b0; #10;
        repeat (1) begin
            data_byte_sent = 1'b1; #10;
            data_byte_sent = 1'b0; #10;
        end
        #100;
        $display("size 0: rows_cnt = %h (expected 01)", rows_cnt);

        // Scenario 2: size 32, 1024 data bytes - expect rows_cnt = 20 (hex).
        size_latched = 2'd1;
        latched = 1'b1; #10;
        latched = 1'b0; #10;
        repeat (1024) begin
            data_byte_sent = 1'b1; #10;
            data_byte_sent = 1'b0; #10;
        end
        #100;
        $display("size 1: rows_cnt = %h (expected 20)", rows_cnt);

        // Scenario 3: size 128, 16384 data bytes - expect rows_cnt = 80 (hex).
        size_latched = 2'd2;
        latched = 1'b1; #10;
        latched = 1'b0; #10;
        repeat (16384) begin
            data_byte_sent = 1'b1; #10;
            data_byte_sent = 1'b0; #10;
        end
        #100;
        $display("size 2: rows_cnt = %h (expected 80)", rows_cnt);

        // Scenario 4: size 256, 65536 data bytes - expect rows_cnt to wrap to 00.
        size_latched = 2'd3;
        latched = 1'b1; #10;
        latched = 1'b0; #10;
        repeat (65536) begin
            data_byte_sent = 1'b1; #10;
            data_byte_sent = 1'b0; #10;
        end
        #100;
        $display("size 3: rows_cnt = %h (expected 00)", rows_cnt);

        $finish;
    end

endmodule
```

Scenario	Data bytes	Expected rows_cnt
1	1 (size 1)	01
2	1,024 (size 32)	20
3	16,384 (size 128)	80
4	65,536 (size 256)	00 (wraps)

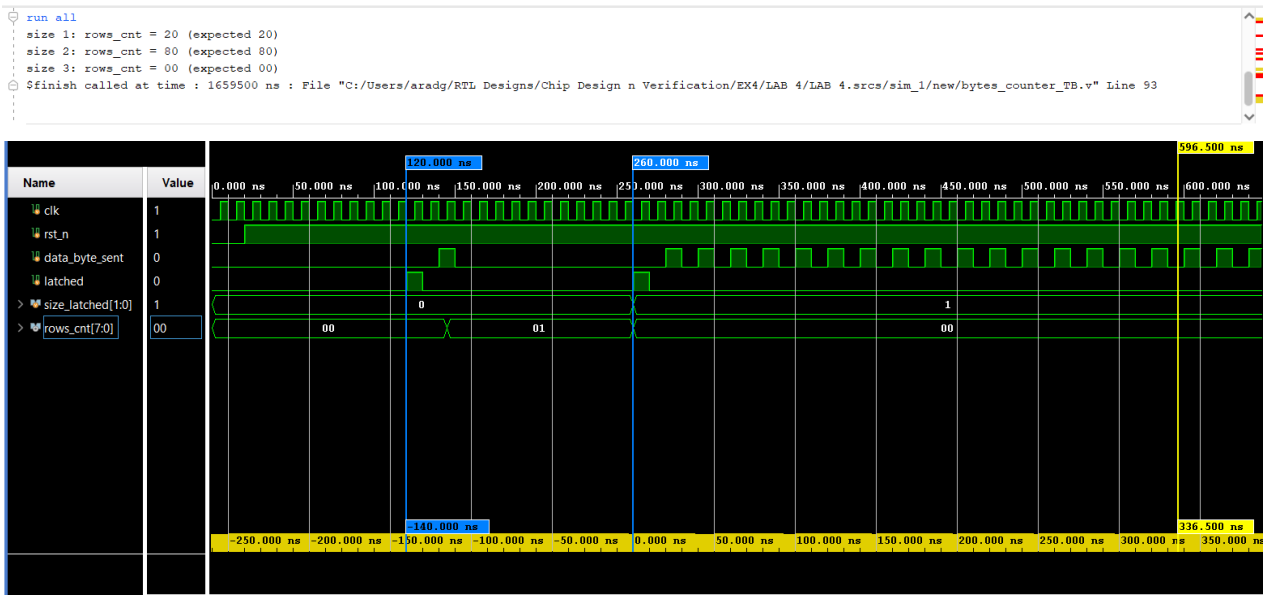


Figure 7 - bytes_counter: rows = bytes / N for all four sizes

בהודעות הקונסולה rows_cnt מגיע ל-20, 80 ו-00 בשלושת הגדלים הגדולים, בדיוק כמו בטבלה. התוצאה של בית בודד, 01, הודפסה עוד לפני ה-run all ולכן לא מופיעה בצילום. בהגדלה, המרקר ב-120ns מראה פולס אחד של data_byte_sent שמעלה את rows_cnt ל-01, והמרקר ב-260ns מראה שה-latched של התרחיש הבא מאפס את המונה ל-00. בגודל 32, rows_cnt עולה רק אחרי 32 תווי נתונים, כי רק אז נספרה שורה שלמה.

seg7_controller_TB

seg7_controller_TB.v

```
`timescale 1ns / 1ps

module seg7_controller_TB;

    reg clk, rst_n;
    reg refresh_tick;
    reg [7:0] char_latched;
    reg [1:0] delay_latched;
    reg [1:0] size_latched;
     reg [7:0] rows_cnt;
     reg valid_latched;
     wire [7:0] an;
     wire [7:0] seg;

    seg7_controller seg7_controller_TB_DUT (
        .clk(clk),
        .rst_n(rst_n),
        .refresh_tick(refresh_tick),
        .char_latched(char_latched),
        .delay_latched(delay_latched),
        .size_latched(size_latched),
        .rows_cnt(rows_cnt),
        .valid_latched(valid_latched),
        .an(an),
        .seg(seg)
    );

    always #5 clk = ~clk;

    // Fast refresh tick for simulation: 10ns pulse every 40ns
    always begin
        #30 refresh_tick = 1'b1;
        #10 refresh_tick = 1'b0;
```

```
    end

    initial begin
        clk = 1'b0;
        rst_n = 1'b0;
        refresh_tick = 1'b0;
        char_latched = 8'h00;
        delay_latched = 2'd0;
        size_latched = 2'd0;
        rows_cnt = 8'h00;
        valid_latched = 1'b0;
        #20;
        rst_n = 1'b1;

        // Scenario 1: nothing latched yet - expect a blank display (an = FF).
        #400;

        // Scenario 2: char 41, delay 0.5, size 20, rows 20 - expect the digits to cycle with DP on an3.
        char_latched = 8'h41;
        delay_latched = 2'd1;
        size_latched = 2'd1;
        rows_cnt = 8'h20;
        valid_latched = 1'b1;
        #400;

        // Scenario 3: char 5A, delay 2.0, size FF, rows FF - expect all digits updated.
        char_latched = 8'h5A;
        delay_latched = 2'd3;
        size_latched = 2'd3;
        rows_cnt = 8'hFF;
        #400;

        // Scenario 4: valid_latched cleared - expect the display to go blank again.
        valid_latched = 1'b0;
        #400;

        $finish;
    end

endmodule
```

Scenario	Inputs	Expected
1	valid_latched = 0	an = FF (blank)
2	char 41, delay 0.5, size 20, rows 20	Digits cycle, DP only on an3
3	char 5A, delay 2.0, size FF, rows FF	All digits updated
4	valid_latched = 0 again	Blank again

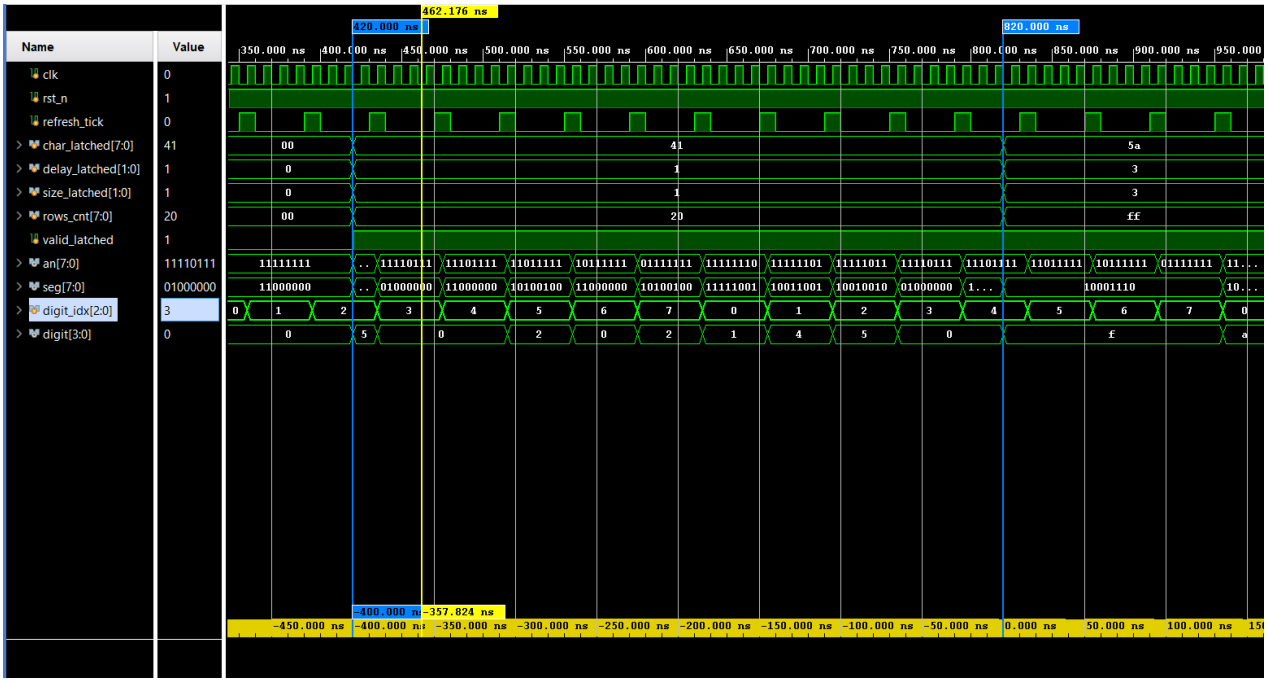


Figure 8 - seg7_controller: one full scan of the 8 digits

הסבב לא מתחיל מספרה 0, כי digit_idx רץ כל הזמן ולא מתאפס כשהערכים משתנים, ולכן קוראים כל ספרה לפי digit_idx. בתרחיש 2, עם 41, 0.5, 20 ו-20, הספרות 0 עד 7 הן 1, 4, 5, 0, 0, 2, 0 ו-2. בכל רגע רק ביט אחד של an הוא 0. במרקר הצהוב digit_idx = 3 ו- seg = 01000000: הביט העליון, DP, הוא 0, כלומר הנקודה דולקת, ושאר הביטים הם הספרה 0, כך שמוצג 0.5.

UART_TX_Top_TB

בדיקה של כל המערכת יחד, כמו משתמש אמיתי: קובעים מתגים, לוחצים על הכפתור, ובודקים מה יוצא בקו ה-UART ובתצוגה. כל הפרמטרים מוקטנים: debounce של 5 מחזורים, ביט של 10 מחזורים, השהיות של 5, 10 ו-20 מחזורים ורענון כל 4 מחזורים.

UART_TX_Top_TB.v

```
1  `timescale 1ns / 1ps
2
3  module UART_TX_Top_TB;
4
5      reg clk, rst_n;
6      reg btn_c;
7      reg [15:0] sw;
8      wire uart_tx;
9      wire led_byte, led_done;
10     wire [7:0] an;
11     wire [7:0] seg;
12
13     always #5 clk = ~clk;
14
15     UART_TX_TOP #(.BTN_CYCLES(5), .BIT_CYCLES(10), .DELAY_50MS(5), .DELAY_100MS(10),   // short timing parameters for simulation
16                   .DELAY_200MS(20),.REFRESH_MAX(3))
17                    UART_TX_TOP_DUT(
18
19          .clk(clk),
20          .rst_n(rst_n),
21          .btn_c(btn_c),
22          .sw(sw),
23          .uart_tx(uart_tx),
24          .led_byte(led_byte),
25          .led_done(led_done),
```

```
26        .an(an),
27        .seg(seg)
28    );
29
30    initial begin
31
32        // Reset and idle preparation
33        clk = 1'b0;
34        rst_n = 1'b0;
35        btn_c = 1'b0;
36        sw = 16'b0;#20;
37        #100;
38
39        rst_n = 1'b1;
40        #100;
41        @(negedge clk);
42        // Scenario 1: single char 'A' (size 1, no delay) - expect one UART frame and led_done set.
43        sw = 16'h0041;
44        btn_c = 1'b1; #100;
45        btn_c = 1'b0;
46        @(posedge led_done);
47        #1000;
48
49        @(negedge clk);
50        // Scenario 2: 32x32 square of 'A' with 50 ms delay - expect 32 rows separated by spaces and CR/LF.
51        sw = 16'h2141;
52        btn_c = 1'b1; #100;
53        btn_c = 1'b0;
54        #50000;
55        // Scenario 3: new press with other switches during transmission - expect it to be ignored.
56        sw = 16'h6342;
57        btn_c = 1'b1; #100;
58        btn_c = 1'b0;
59        @(posedge led_done);
60        #1000;
61
62        @(negedge clk);
63        // Scenario 4: reset during a 32x32 transmission - expect it to stop, display blank and LEDs off.
64        sw = 16'h2141;
65        btn_c = 1'b1; #100;
66        btn_c = 1'b0;
67        #1000000;
68        rst_n = 1'b0; #30;
69        rst_n = 1'b1;
70        #1000;
71
72        $finish;
73    end
74endmodule
```

Scenario	Test	Expected
1	Single 'A' (sw = 0041)	One UART frame, led_done = 1
2	32x32 of 'A', 50 ms (sw = 2141)	32 rows, spaces and CR+LF
3	New press during scenario 2 (sw = 6342)	Ignored, display unchanged
4	Reset during 32x32	Stops, display blank, LEDs off

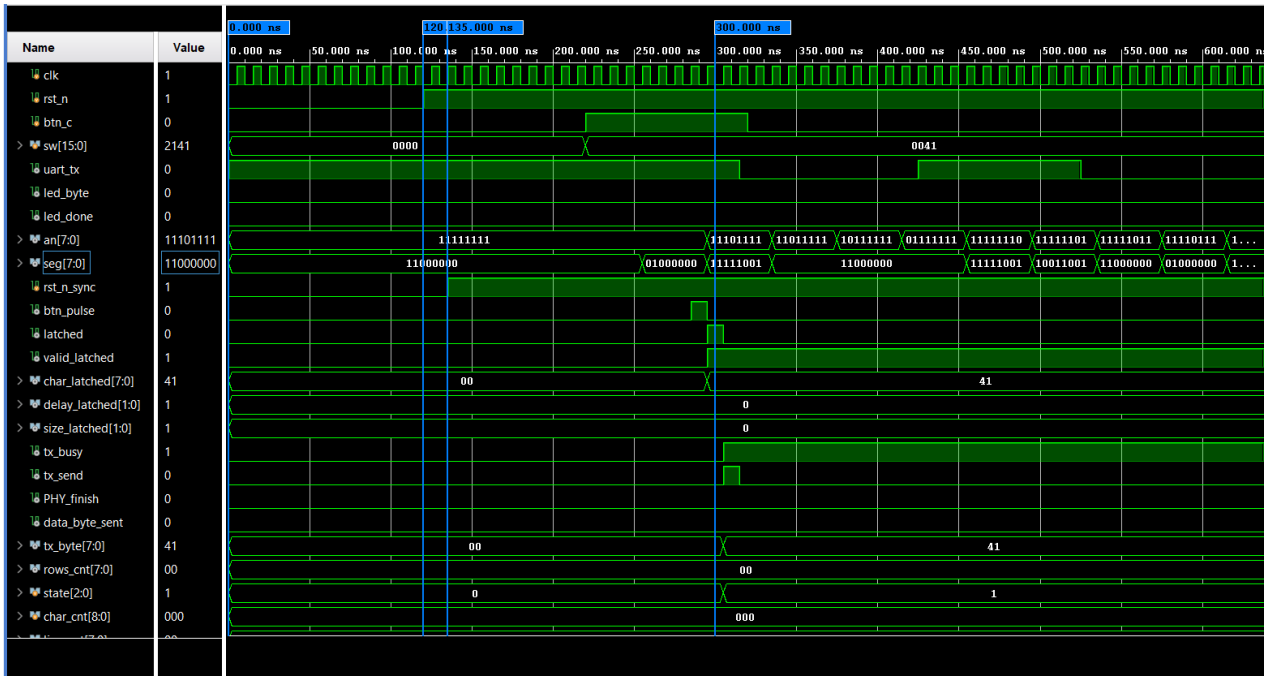


Figure 9 - Top level: reset and preparation

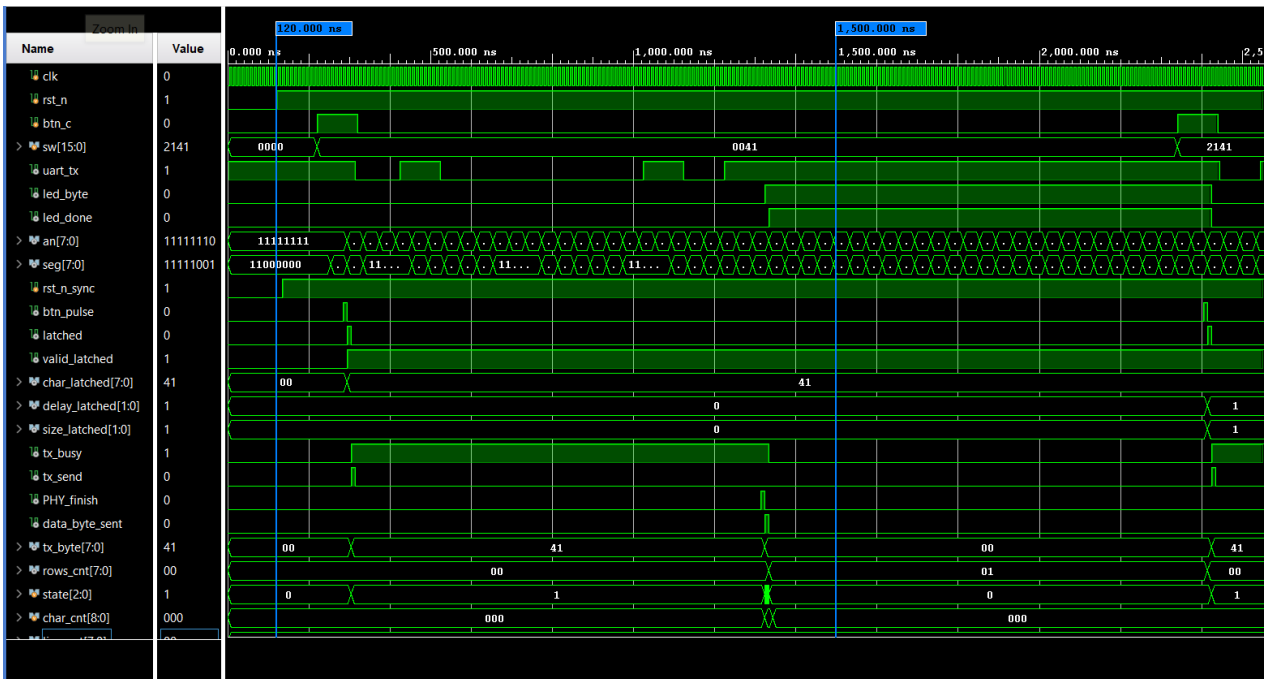


Figure 10 - Top level, scenario 1: a single character

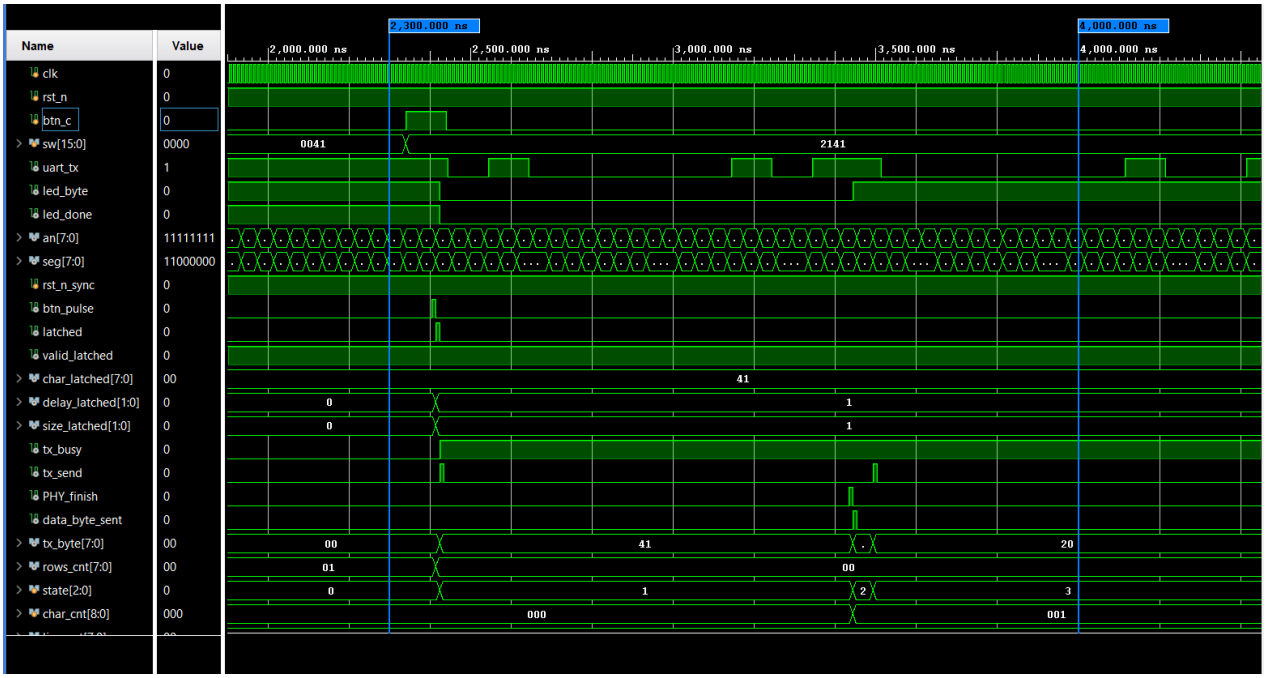


Figure 11 - Top level, scenario 2: start of the square

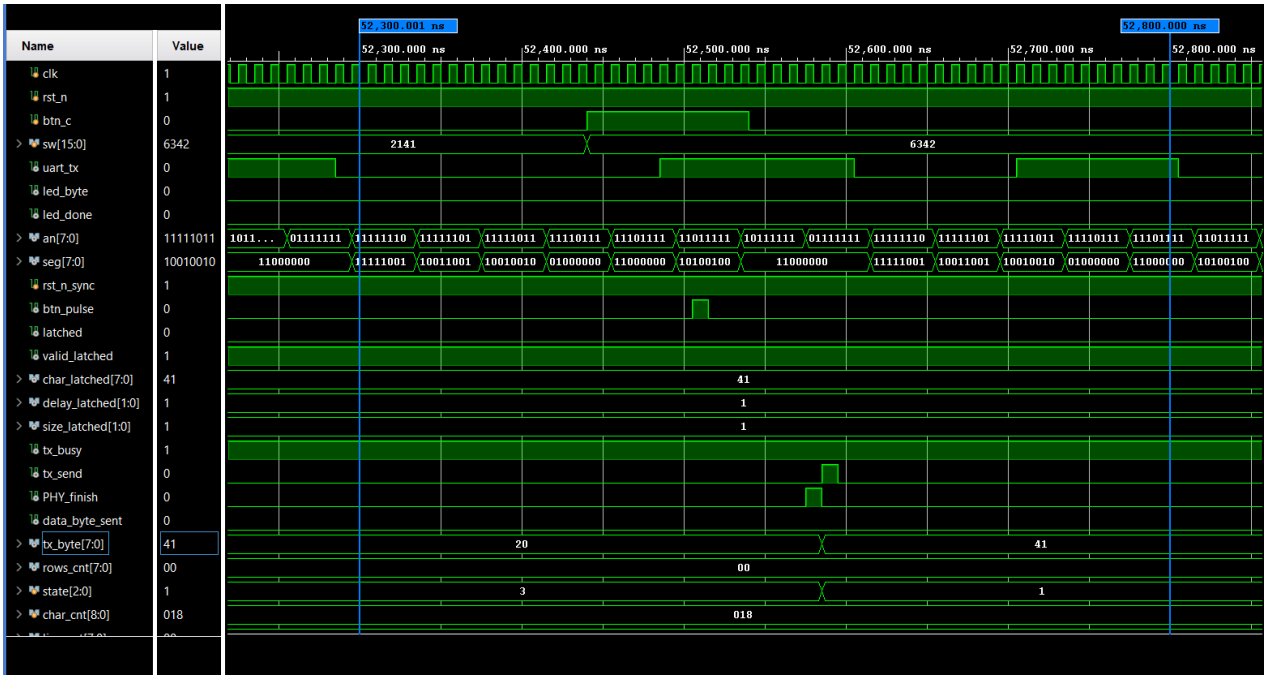


Figure 12 - Top level, scenario 3: a press during transmission is ignored

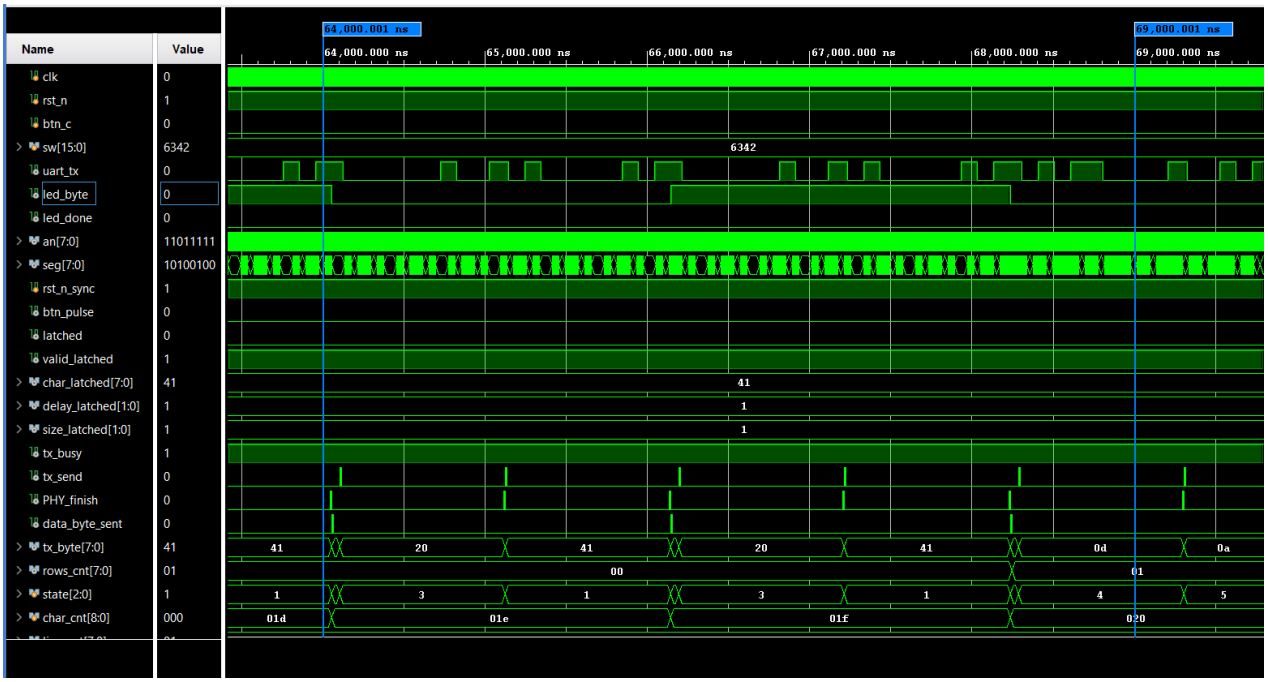


Figure 13 - Top level: end of a row

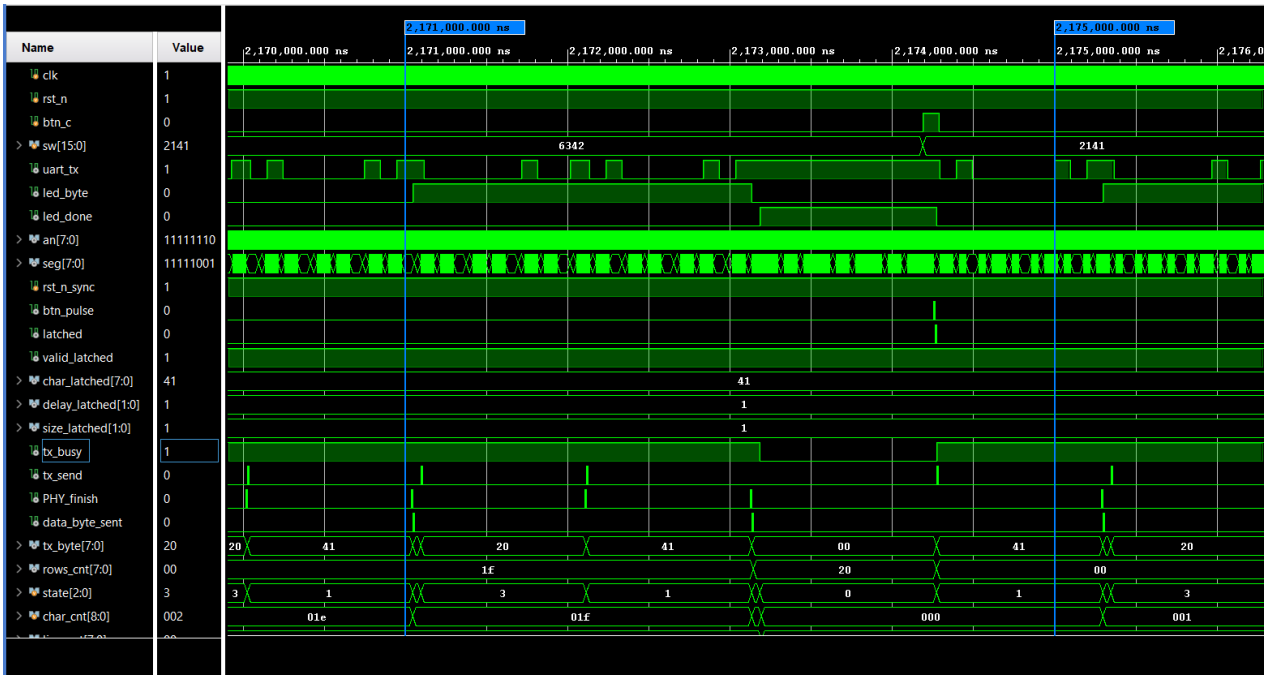


Figure 14 - Top level: end of scenario 2

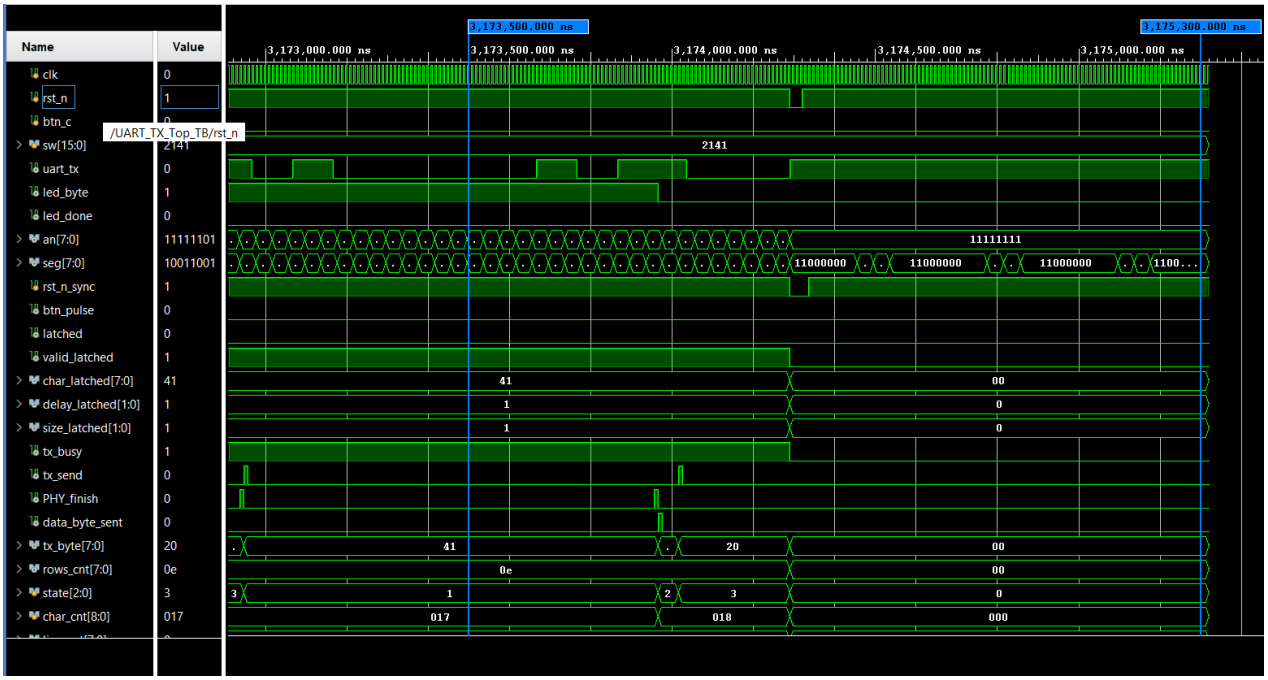


Figure 15 - Top level, scenario 4: reset during transmission

Step 5 - Physical Implementation

קובץ האילוצים מבוסס על ה-XDC Master של Digilent ללוח Nexys A7-100T. הפעלתי רק את השורות של הרכיבים שבתכן, ובכל שורה כזו שיניתי את שם הפורט לשם שבקוד. השורות של LED[1] עד LED[14], של שאר הכפתורים ושל הממשקים האחרים נשארו כהערות.

Board pin name	Port in UART_TX_TOP	Pin
CLK100MHZ	clk	E3
CPU_RESETN	rst_n	C12
BTNC	btn_c	N17
SW[0]-SW[15]	sw[0]-sw[15]	J15 ... V10
UART_RXD_OUT	uart_tx	D4
LED[0]	led_byte	H17
LED[15]	led_done	V11
CA-CG, DP	seg[0]-seg[6], seg[7]	T10 ... L18, H15
AN[0]-AN[7]	an[0]-an[7]	J17 ... U13

שם הפין של ה-UART: בקובץ של Digilent הפין D4 נקרא UART_RXD_OUT. השם נכתב מנקודת המבט של שבב הגשר USB-UART שעל הלוח: זה הקו שהשבב מקבל ממנו (RX), כלומר הקו שה-FPGA משדר בו. לכן הוא מחובר ליציאה uart_tx.

רמות מתח: SW8 ו-SW9 מחוברים בלוח לבנק של 1.8V, ולכן הם מוגדרים כ-LVCMOS18, כמו בקובץ המקורי. שאר הפינים מוגדרים כ-LVCMOS33. הנקודה העשרונית (DP) מחוברת ל-seg[7], ושבעת הקטעים ל-seg[0] עד seg[6] באותו סדר שבו בנויות התבניות במפענח.

Nexys-A7-100T-Master.xdc (active lines)

```
## Clock: 100MHz system clock
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { clk }];
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports {clk}];

## Switches (SW[8] and SW[9] are on a 1.8V bank -> LVCMOS18)
set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { sw[0] }];
set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { sw[1] }];
set_property -dict { PACKAGE_PIN M13   IOSTANDARD LVCMOS33 } [get_ports { sw[2] }];
```

```
set_property -dict { PACKAGE_PIN R15   IOSTANDARD LVCMOS33 } [get_ports { sw[3] }];
set_property -dict { PACKAGE_PIN R17   IOSTANDARD LVCMOS33 } [get_ports { sw[4] }];
set_property -dict { PACKAGE_PIN T18   IOSTANDARD LVCMOS33 } [get_ports { sw[5] }];
set_property -dict { PACKAGE_PIN U18   IOSTANDARD LVCMOS33 } [get_ports { sw[6] }];
set_property -dict { PACKAGE_PIN R13   IOSTANDARD LVCMOS33 } [get_ports { sw[7] }];
set_property -dict { PACKAGE_PIN T8    IOSTANDARD LVCMOS18 } [get_ports { sw[8] }];
set_property -dict { PACKAGE_PIN U8    IOSTANDARD LVCMOS18 } [get_ports { sw[9] }];
set_property -dict { PACKAGE_PIN R16   IOSTANDARD LVCMOS33 } [get_ports { sw[10] }];
set_property -dict { PACKAGE_PIN T13   IOSTANDARD LVCMOS33 } [get_ports { sw[11] }];
set_property -dict { PACKAGE_PIN H6    IOSTANDARD LVCMOS33 } [get_ports { sw[12] }];
set_property -dict { PACKAGE_PIN U12   IOSTANDARD LVCMOS33 } [get_ports { sw[13] }];
set_property -dict { PACKAGE_PIN U11   IOSTANDARD LVCMOS33 } [get_ports { sw[14] }];
set_property -dict { PACKAGE_PIN V10   IOSTANDARD LVCMOS33 } [get_ports { sw[15] }];

## LEDs: LD0 = led_byte, LD15 = led_done
set_property -dict { PACKAGE_PIN H17   IOSTANDARD LVCMOS33 } [get_ports { led_byte }];
set_property -dict { PACKAGE_PIN V11   IOSTANDARD LVCMOS33 } [get_ports { led_done }];

## 7-segment cathodes and DP (active low): seg[0] = CA ... seg[6] = CG, seg[7] = DP
set_property -dict { PACKAGE_PIN T10   IOSTANDARD LVCMOS33 } [get_ports { seg[0] }];
set_property -dict { PACKAGE_PIN R10   IOSTANDARD LVCMOS33 } [get_ports { seg[1] }];
set_property -dict { PACKAGE_PIN K16   IOSTANDARD LVCMOS33 } [get_ports { seg[2] }];
set_property -dict { PACKAGE_PIN K13   IOSTANDARD LVCMOS33 } [get_ports { seg[3] }];
set_property -dict { PACKAGE_PIN P15   IOSTANDARD LVCMOS33 } [get_ports { seg[4] }];
set_property -dict { PACKAGE_PIN T11   IOSTANDARD LVCMOS33 } [get_ports { seg[5] }];
set_property -dict { PACKAGE_PIN L18   IOSTANDARD LVCMOS33 } [get_ports { seg[6] }];
set_property -dict { PACKAGE_PIN H15   IOSTANDARD LVCMOS33 } [get_ports { seg[7] }];

## 7-segment anodes (active low)
set_property -dict { PACKAGE_PIN J17   IOSTANDARD LVCMOS33 } [get_ports { an[0] }];
set_property -dict { PACKAGE_PIN J18   IOSTANDARD LVCMOS33 } [get_ports { an[1] }];
set_property -dict { PACKAGE_PIN T9    IOSTANDARD LVCMOS33 } [get_ports { an[2] }];
set_property -dict { PACKAGE_PIN J14   IOSTANDARD LVCMOS33 } [get_ports { an[3] }];
set_property -dict { PACKAGE_PIN P14   IOSTANDARD LVCMOS33 } [get_ports { an[4] }];
set_property -dict { PACKAGE_PIN T14   IOSTANDARD LVCMOS33 } [get_ports { an[5] }];
set_property -dict { PACKAGE_PIN K2    IOSTANDARD LVCMOS33 } [get_ports { an[6] }];
set_property -dict { PACKAGE_PIN U13   IOSTANDARD LVCMOS33 } [get_ports { an[7] }];

## CPU reset (active low) and center button
set_property -dict { PACKAGE_PIN C12   IOSTANDARD LVCMOS33 } [get_ports { rst_n }];
set_property -dict { PACKAGE_PIN N17   IOSTANDARD LVCMOS33 } [get_ports { btn_c }];

## USB-UART bridge: FPGA transmit line
set_property -dict { PACKAGE_PIN D4    IOSTANDARD LVCMOS33 } [get_ports { uart_tx }];
```

Board test

אחרי יצירת ה-Bitstream צרבתי את הלוח, וחיברתי אותו למחשב דרך PuTTY בחיבור סריאלי: COM7, 57600 baud, 8 ביטי נתונים, ללא parity, ביט עצירה אחד וללא flow control. את הפלט של כל תרחיש שמרתי לקובץ לוג בעזרת אפשרות ה-Logging של PuTTY. כל התרחישים עבדו:

#	Switches	Action	Result
1	0041 (A, 0 ms, 1x1)	Press BTNC	A single 'A', display 41 / 0.0 / 01 / 01, LD15 on
2	2141 (A, 50 ms, 32x32)	Press BTNC	32 rows, rows counter 01 to 20, LD0 toggles, LD15 at the end
3	Change switches during scenario 2	Press BTNC	Ignored, display and output unchanged
4	6041 (A, 0 ms, 256x256)	Press BTNC	256 rows in about 23 s, display 41 / 0.0 / FF / 00, LD15 on
5	2142 (B, 50 ms, 32x32)	Press BTNC, then CPU RESET	Transmission stops, display blank, LEDs off

תרחיש 1, תו בודד: נשלח תו אחד, בלי רווח ובלי ירידת שורה. התצוגה מראה תו 41, השהיה 0.0, גודל 01 ושורה 01, ושני הלדים דולקים: led_done בסוף השידור, ו-led_byte שהתהפך פעם אחת.

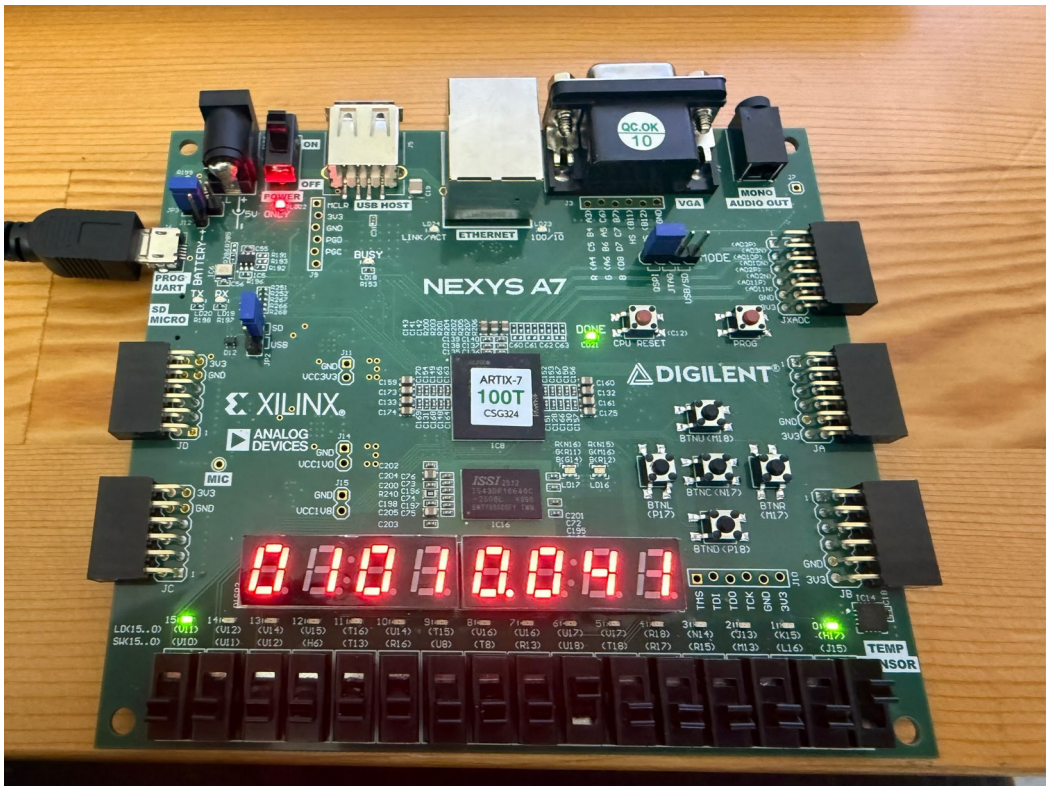


Figure B1 - Board, scenario 1: a single 'A', rows 01, size 01, delay 0.0, LD15 and LD0 on

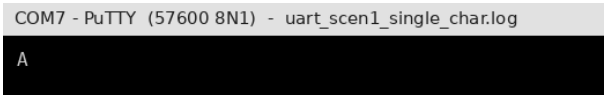


Figure B2 - PuTTY log, scenario 1: one character

תרחיש 2, ריבוע של 32 עם השהיה של 50 ms: מונה השורות עולה בהדרגה, led_byte מהבהב עם כל תו, ובסוף התצוגה מראה 20 (32 בהקס) ו-led_done נדלק. השידור לקח קצת פחות מדקה, כמו שחושב בשלב הדרישות.

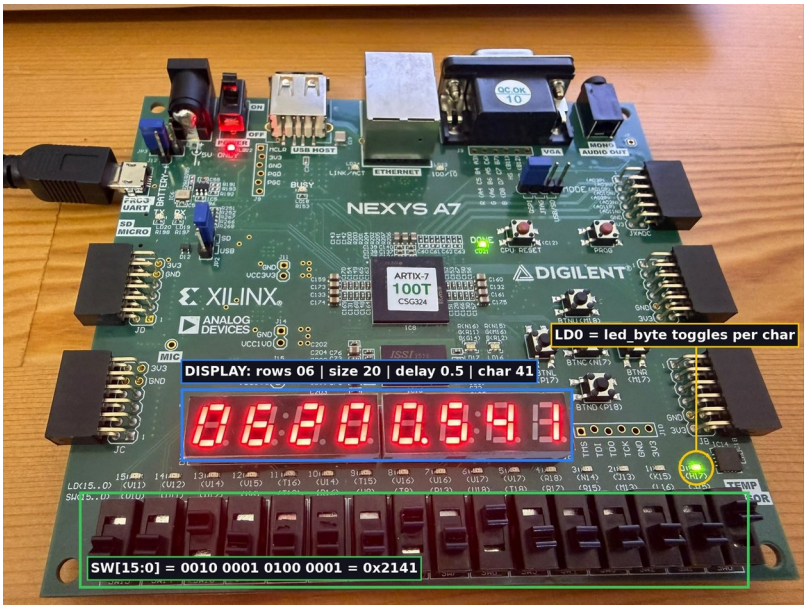


Figure B3 - Board, scenario 2: during transmission, rows 06, LD0 toggling

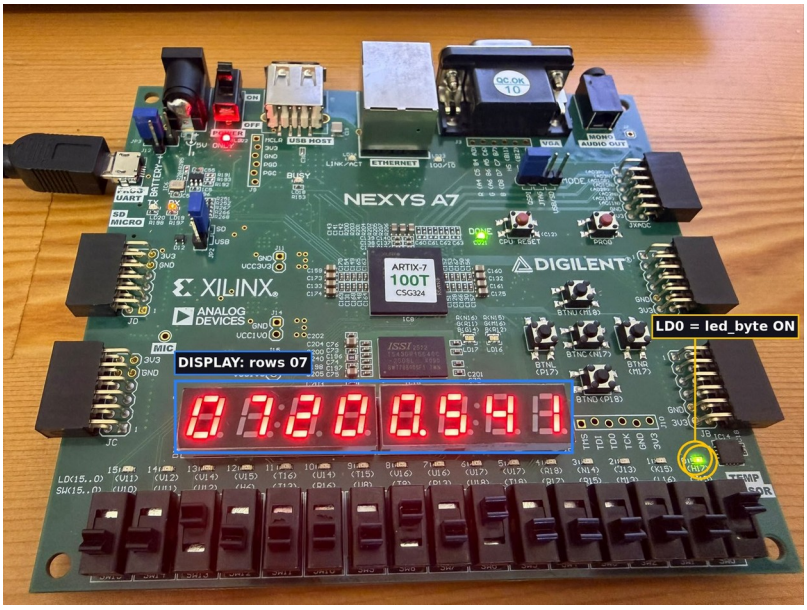


Figure B4 - Board, scenario 2: rows 07

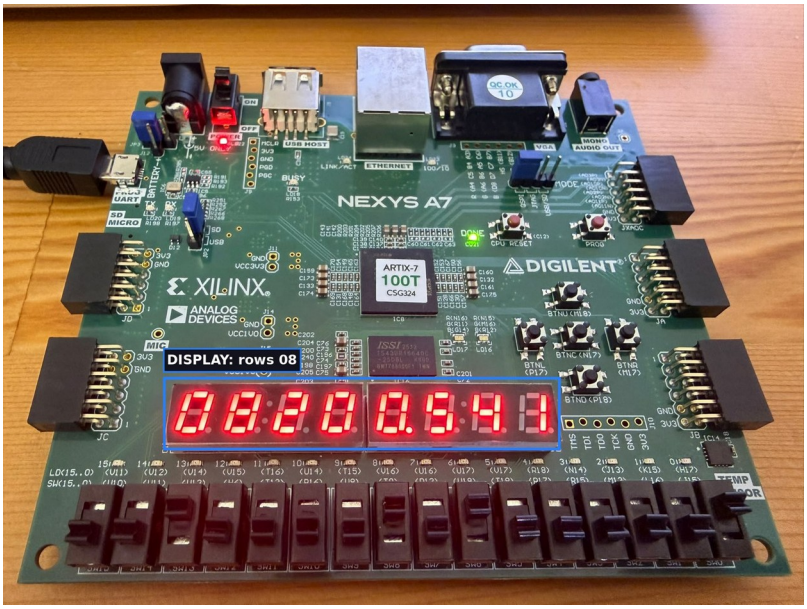


Figure B5 - Board, scenario 2: rows 08

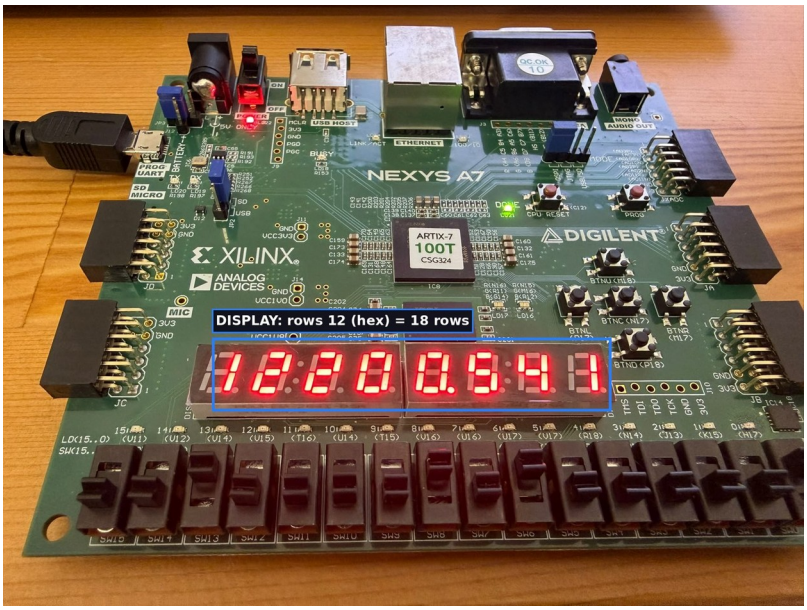


Figure B6 - Board, scenario 2: rows 12 (hex), 18 rows sent

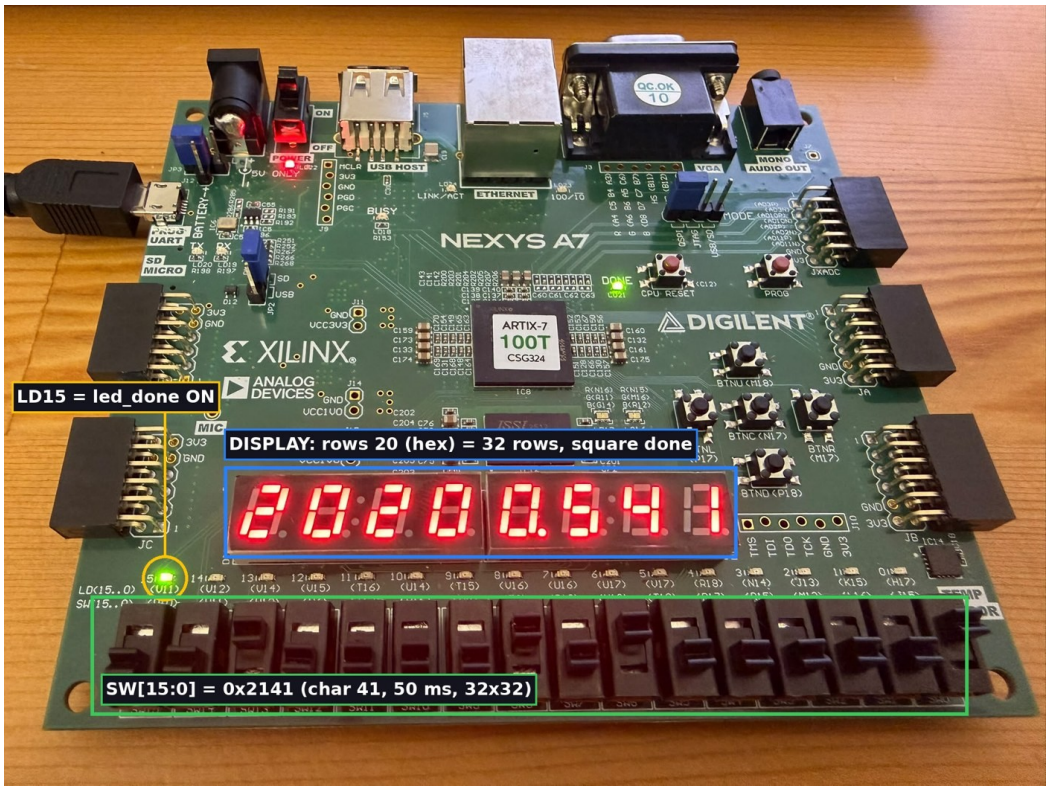


Figure B7 - Board, scenario 2 done: rows 20 (hex) = 32 rows, LD15 = led_done on

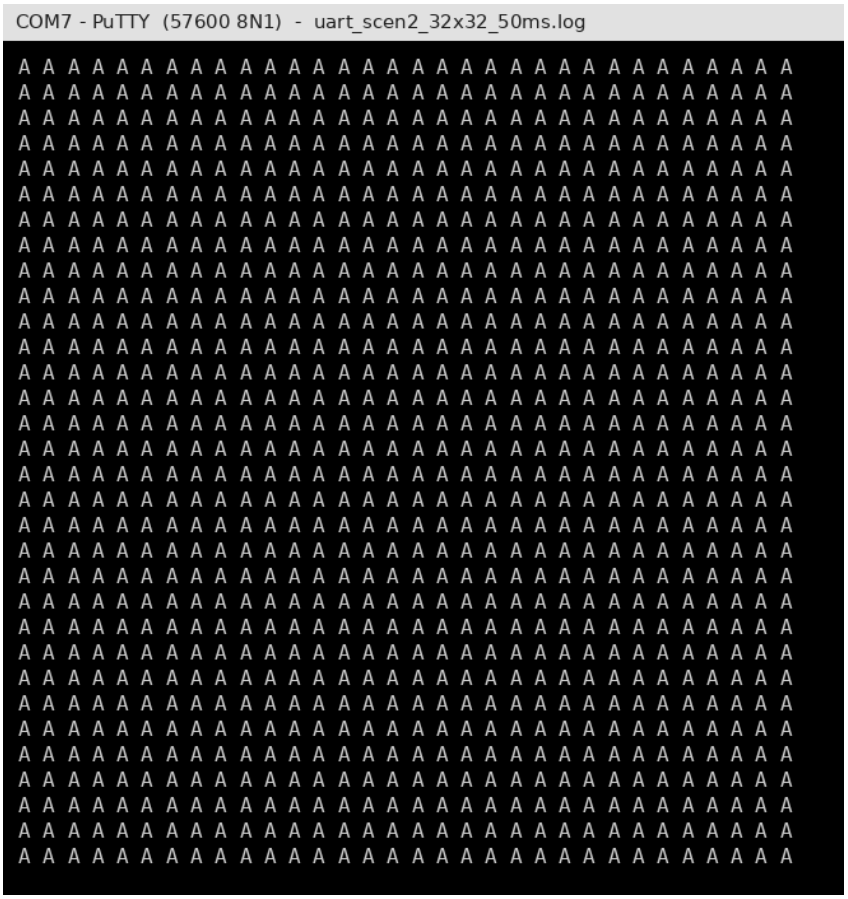


Figure B8 - PuTTY log, scenario 2: 32 rows of 32 'A' characters separated by spaces

תרחיש 3, לחיצה באמצע שידור: בזמן שתרחיש 2 רץ שיניתי את המתגים ולחצתי שוב. הערכים בתצוגה לא השתנו, והשידור המשיך עד הסוף בלי הפרעה. אין לתרחיש הזה צילום נפרד.

תרחיש 4, ריבוע של 256 בלי השהיה: השידור לקח כ-23 שניות. בסוף התצוגה מראה גודל FF ושורות 00, כי מונה השורות של 8 ביט עוטף אחרי 255, בדיוק כמו שהוחלט. בטרמינל כל שורה באורך 511 תווים, ולכן החלון שובר אותה. במקום צילום של הטרמינל, הפלט נבדק מתוך קובץ הלוג.

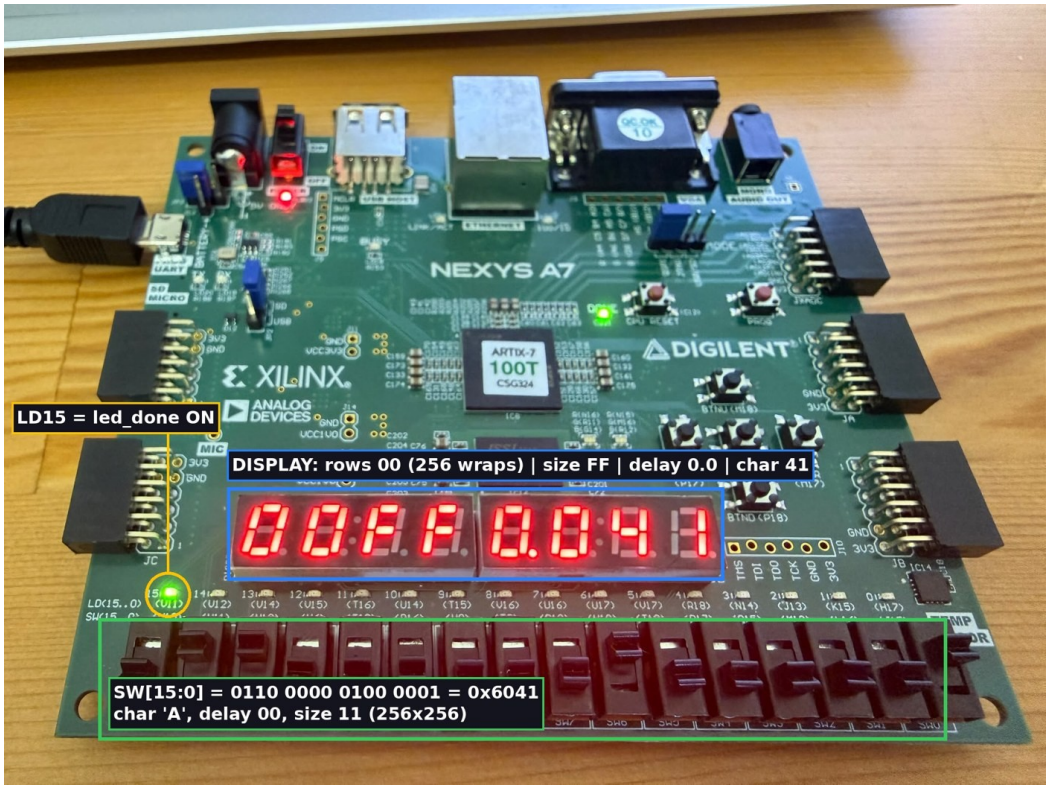
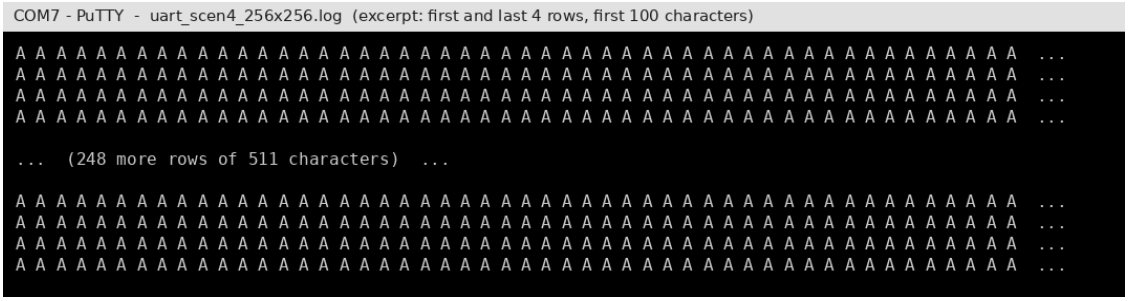


Figure B9 - Board, scenario 4: 256x256 done, size FF, rows wrap to 00, LD15 on


COM7 - PuTTY - uart_scen4_256x256.log (excerpt: first and last 4 rows, first 100 characters)
A A A

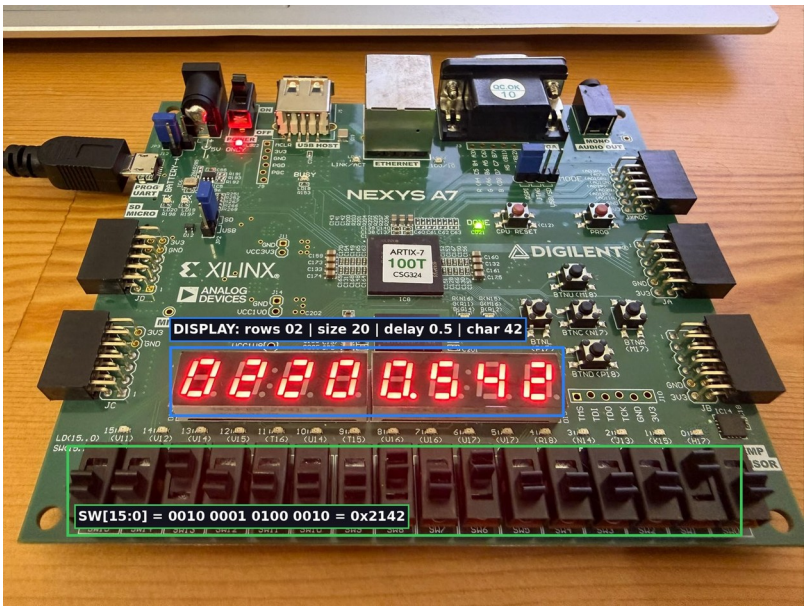


Figure B11 - Board, scenario 5: transmission of 'B' started, rows 02

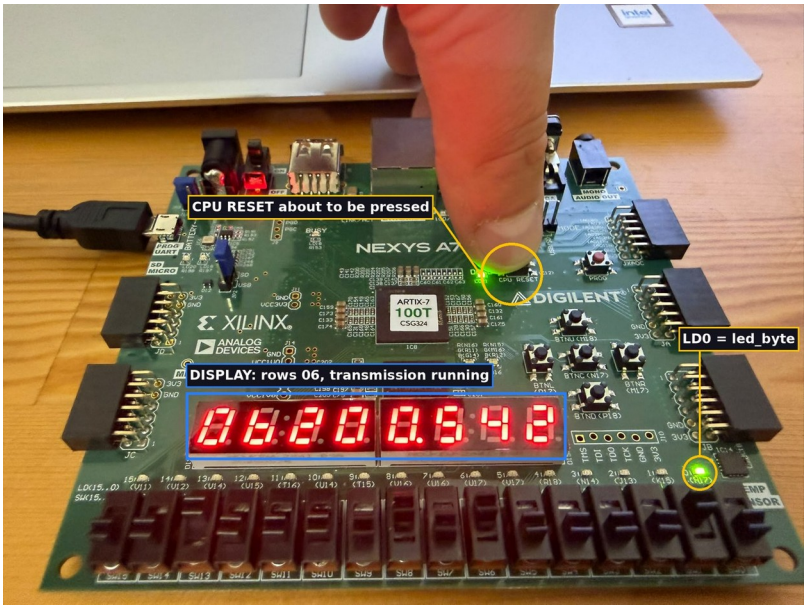


Figure B12 - Board, scenario 5: rows 06, about to press CPU RESET

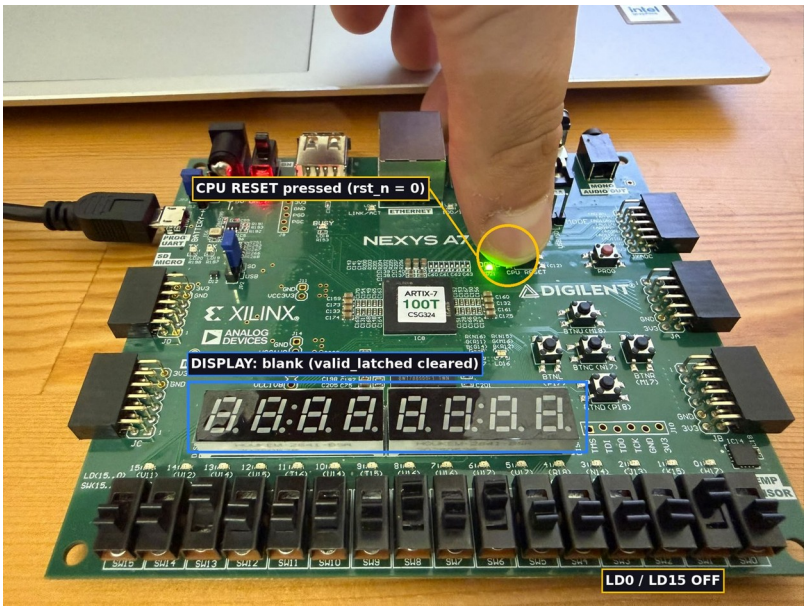


Figure B13 - Board, scenario 5: CPU RESET pressed, display blank and LEDs off

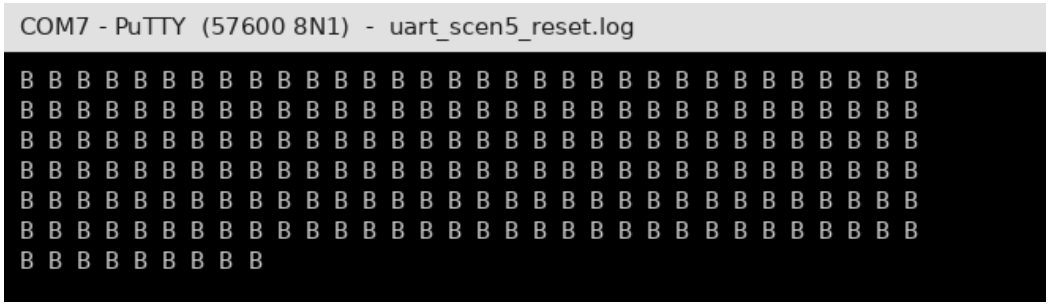


Figure B14 - PuTTY log, scenario 5: 6 full rows and 9 characters, stopped by CPU RESET

בלוג רואים 6 שורות מלאות של 32 תווי B, מופרדים ברווחים, ואחריהן רק 9 תווים בשורה השביעית. זה מתאים לתצוגה באיור B12, שהראתה 06 rows רגע לפני הלחיצה על CPU RESET. השורה האחרונה נקטעה באמצע, בלי CR ו-LF בסופה, כלומר הריסט עצר את השידור מיד ולא נשלח אף תו אחרי הלחיצה. בשידור הבא הסמן יתחיל מאותה נקודה בטרמינל, כי לא נשלחה ירידת שורה.

Step 6 - Synthesis, Implementation & Reports

Lint Report

הרצתי Lint כש-UART_TX_TOP מוגדר כמודול העליון. התקבלו שתי הפרות, ושתיהן מכוונות:

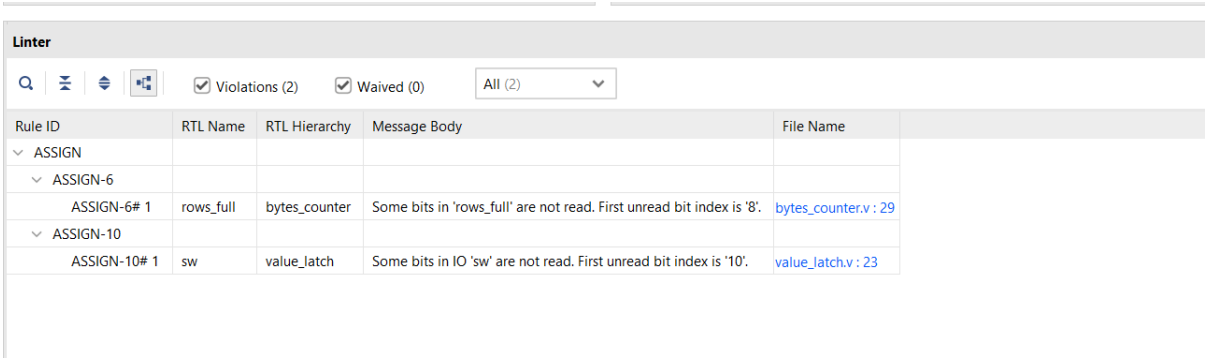


Figure 16 - Linter: two violations before the waivers

- **ASSIGN-6, rows_full ב-bytes_counter:** התוצאה של ההזזה היא ברוחב 17 ביט, אבל לתצוגה נכנסים רק 8 הביטים התחתונים. זה החיתוך המכוון שגורם למונה השורות להציג 00 בסוף ריבוע של 256.
- **ASSIGN-10, sw ב-value_latch:** המתגים sw[10], sw[11], sw[12] ו-sw[15] לא בשימוש לפי האפיון. הלינטר מציין רק את האינדקס הראשון שלא נקרא, 10.

לכל אחת מההפרות יצרתי Waiver עם הצדקה כתובה, ואחרי זה הלינטר נקי: אפס הפרות ושני ויתורים.

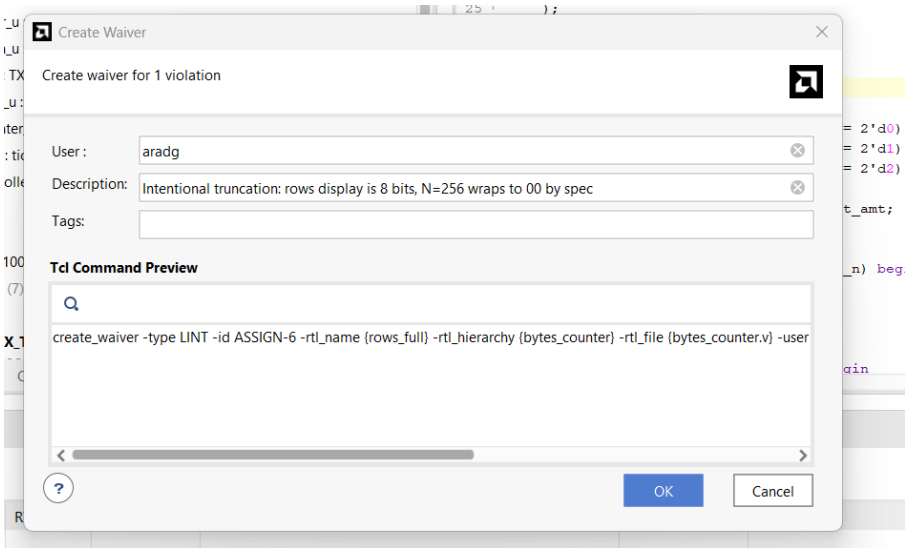


Figure 17 - Waiver for ASSIGN-6 with its justification

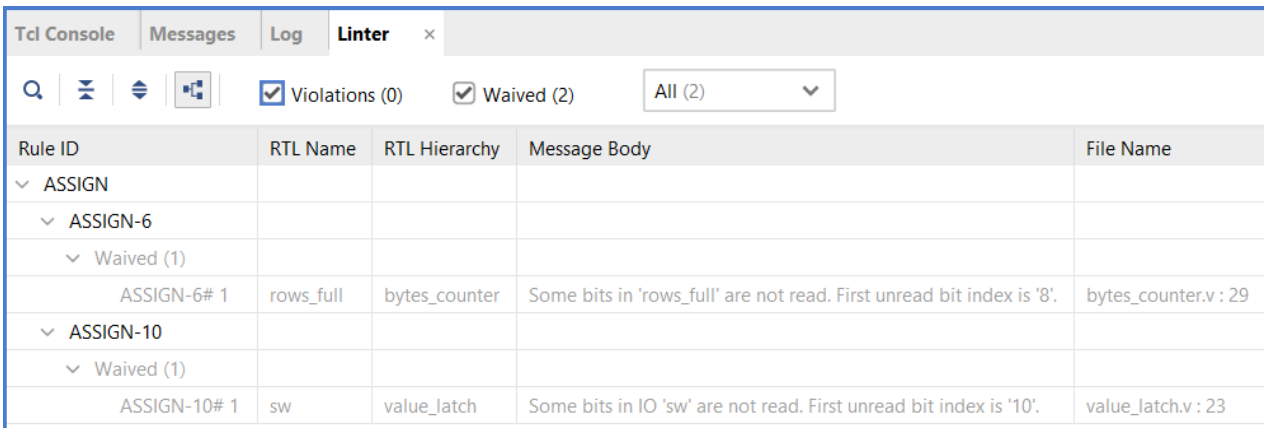


Figure 18 - Linter: no violations, two waivers

RTL Schematic

הסכמה ש-Vivado מייצר מהתכן המעובד (Elaborated Design) מראה את הבלוקים ואת החיבורים ביניהם. היא צריכה להתאים לדיאגרמת הבלוקים מ-Step 2.

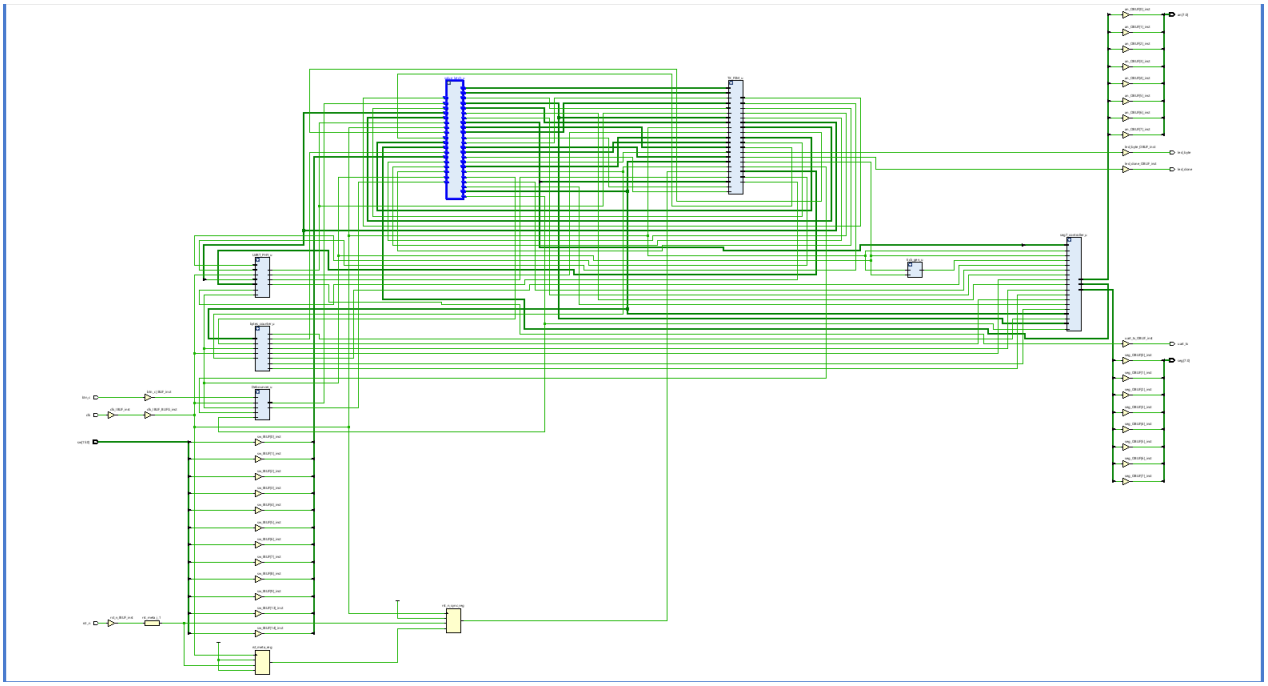


Figure 18b - RTL schematic (Elaborated Design) of UART_TX_TOP

Synthesis & Implementation Messages

- **Synth 8-7129 (8 אזהרות):** פורטים בלי עומס. הן נוגעות לארבעת המתגים שלא בשימוש, sw[10], sw[11], sw[12] ו-sw[15], וכל אחד מהם מופיע פעמיים: פעם ב-value_latch ופעם ב-UART_TX_TOP. כלומר 4 מתגים כפול 2 מודולים, והאזהרות צפויות.
- **קידוד מחדש של המצבים:** Vivado זיהה את מכונת המצבים של TX_FSM וקידד אותה מחדש ל-one-hot. במקום 3 פליפ-פלופים בקידוד בינארי יש 6, אחד לכל מצב. זה מוסיף שלושה פליפ-פלופים, אבל הלוגיקה של המצב הבא והיציאות נעשית פשוטה ומהירה יותר, וב-FPGA יש הרבה פליפ-פלופים.

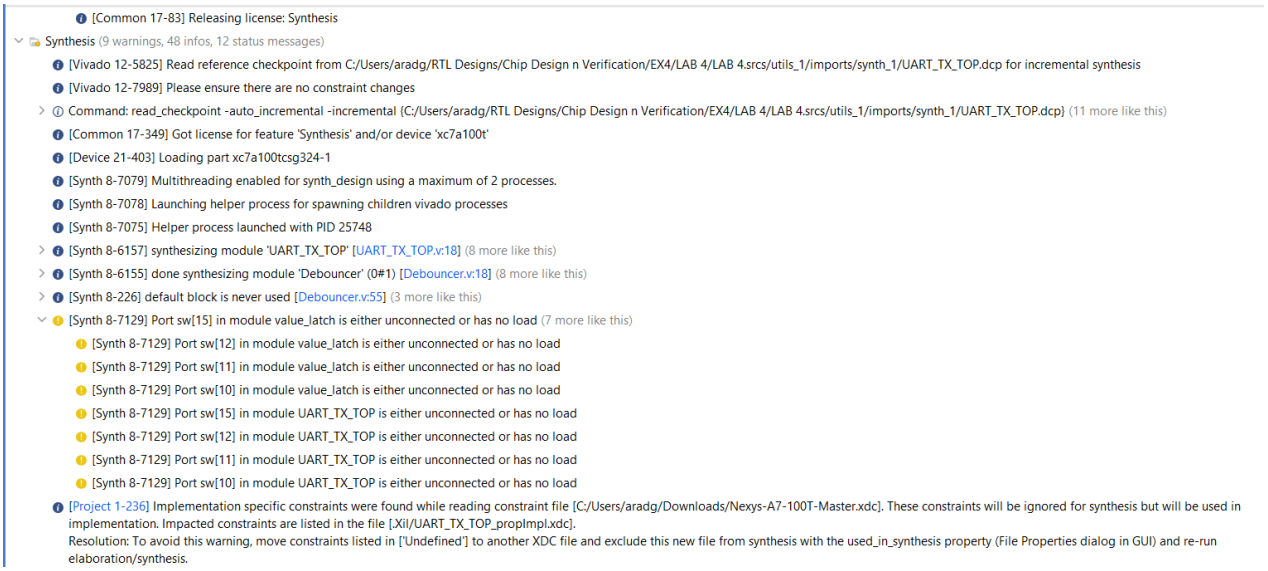


Figure 19 - Synthesis messages after the fix: no critical warnings, the eight Synth 8-7129 warnings expanded

Timing Report

מסמך התרגיל לא מבקש דוח תזמון במפורש, אבל הוא ההוכחה הישירה לכך שה-Implementation נקי. דוח התזמון אחרי Implementation, עם אילוץ שעון של 10ns (100MHz):

Metric	Setup	Hold
Worst Slack	ns 1.761	ns 0.032
Total Negative Slack	ns 0.000	ns 0.000
Failing Endpoints	0	0
Total Endpoints	396	396

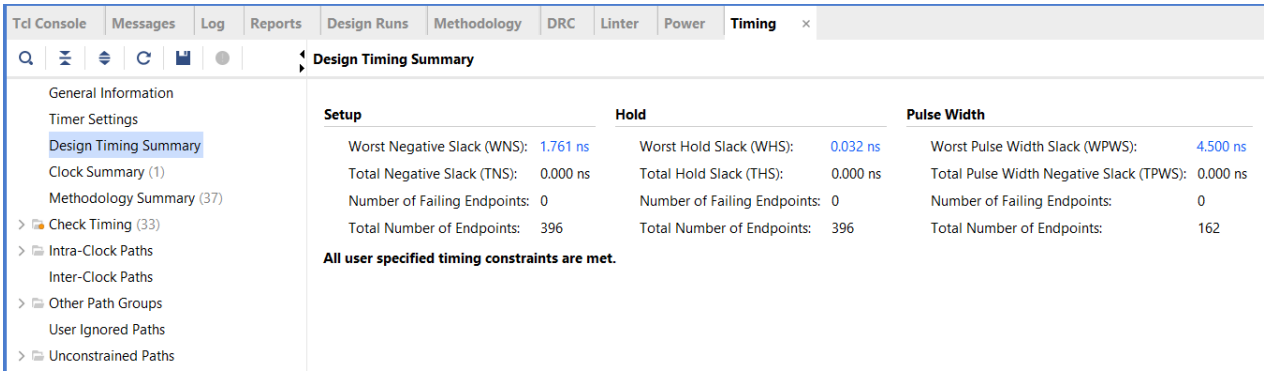


Figure 20 - Implemented design timing summary

שני ה-slack חיוביים, ואין אף endpoint שנכשל, ולכן כל אילוצי התזמון מתקיימים. WNS של 1.761 ns אומר שהנתיב האיטי ביותר בתכן מסתיים 1.761 ns לפני קצה השעון הבא, כך שהתכן היה עומד גם בשעון מהיר יותר, עד כ-121MHz (1 חלקי 8.239 ns). WHS של 0.032 ns אומר שאין הפרות hold.

ב-Check Timing השדות no_clock ו-unconstrained_internal_endpoints הם 0, כלומר כל הרגיסטרים בתכן מקבלים את השעון המאולץ. ההערות שם נוגעות ל-14 כניסות ול-19 יציאות שאין להן אילוצי תזמון (set_input_delay / set_output_delay). המספרים תואמים בדיוק לתכן: 14 הכניסות הן הריסט, הכפתור ו-12 המתגים שבשימוש, ו-19 היציאות הן קו ה-UART, שני הלדים, 8 האנודות ו-8 הקטעים. זה תקין כאן: הכניסות משתנות בלי קשר לשעון ועוברות סנכרון בתוך התכן, והיציאות מחוברות ללדים, לתצוגה ולקו UART איטי, כך שאין להן דרישת תזמון מול השעון.

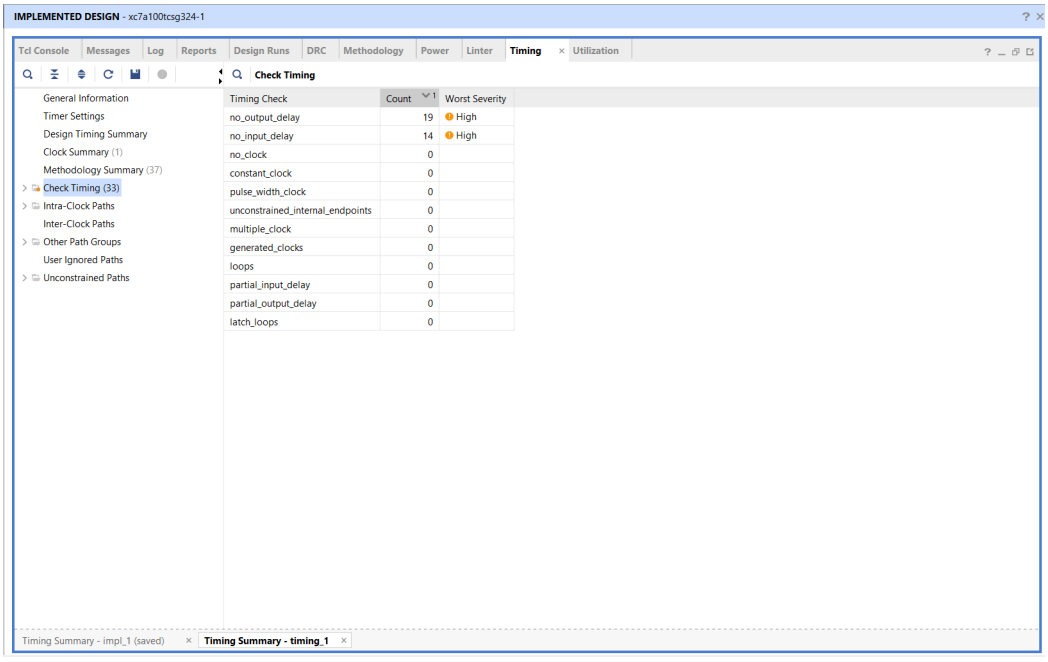


Figure 21 - Check Timing: no unclocked registers, only unconstrained I/O

DRC

דוח ה-DRC מכיל אזהרה אחת, CFGBVS-1: בתכן לא מוגדרים המאפיינים CFGBVS ו-CONFIG_VOLTAGE, שמתארים את מתח בנק התצורה. האזהרה לא משפיעה על פעולת התכן, וזה גם מה שראינו על הלוח. כדי לנקות אותה מוסיפים לקובץ האילוצים את השורות set_property CFGBVS VCCO [current_design] ו-set_property CONFIG_VOLTAGE 3.3 [current_design].

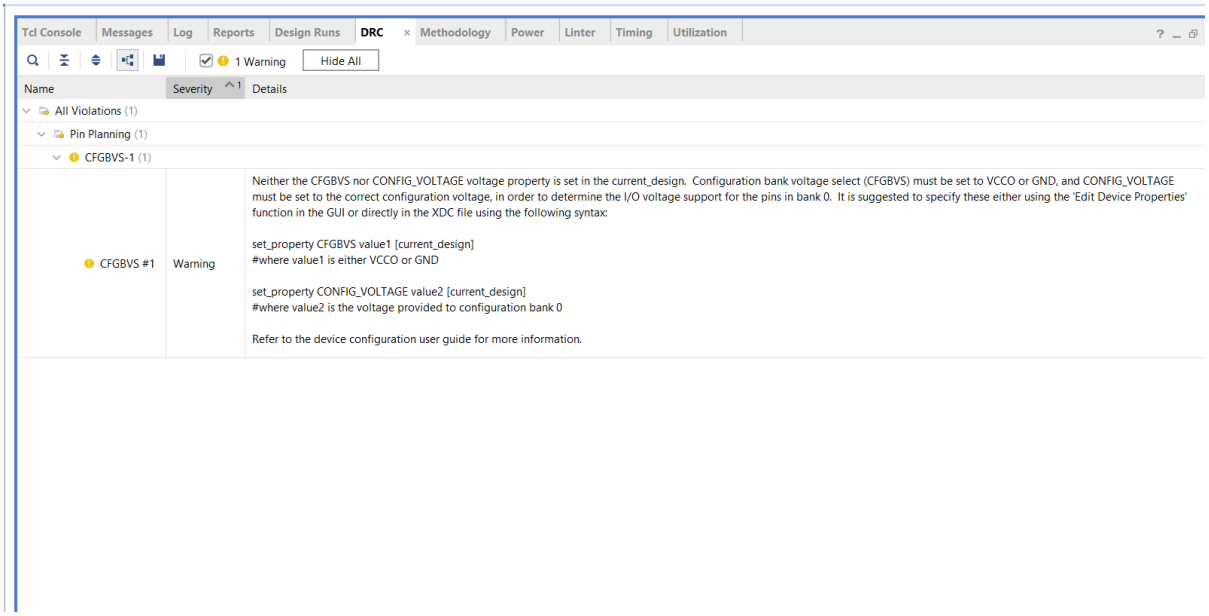


Figure 22 - DRC: a single CFGBVS-1 warning

Step 7 - Resource Expectations vs. Utilization

לפני הסינתזה הערכתי כמה פליפ-פלופים התכן צריך, לפי הביטים של כל רגיסטר:

Block	Registers	Bits
UART_TX_TOP	rst_meta, rst_n_sync	2
Debouncer	2 sync FFs, stable_cnt (21), state	24
value_latch	char (8), delay (2), size (2), latched, valid_latched	14
UART_PHY	uart_tx, PHY_finish, shift_reg (8), cycle_cnt (11), index_cnt (4), state	26
TX_FSM	state (3), tx_byte (8), 5 control outputs, delay_cnt (25), char_cnt (9), line_cnt (8)	58
bytes_counter	byte_cnt	17
tick_gen	tick_cnt	15
seg7_controller	digit_idx	3
Total		159

את מספר ה-LUTs קשה להעריך מראש, כי הוא תלוי באופן שבו הסינתזה בונה את המשווים, המונים והמרבבים. ציפיתי למאות בודדות לכל היותר. אין בתכן המרה ל-BCD, שבמעבדה 3 תפסה את רוב ה-LUTs.

Resource	Expected	Used	Available	Util %
Slice Registers	159	161	126800	0.13%
Slice LUTs	a few hundred	206	63400	0.32%
Bonded IOB	34	34	210	16.2%

Resource	Expected	Used	Available	Util %
BUFGCTRL	1	1	32	3.1%

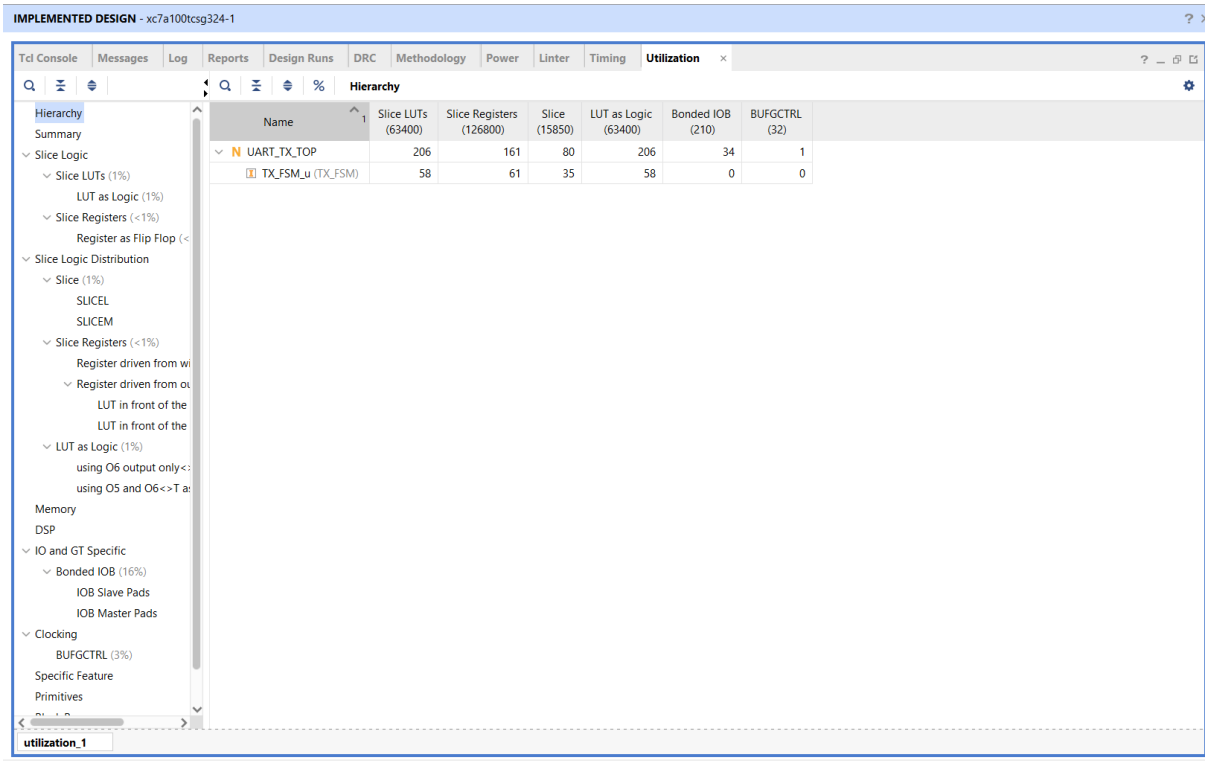


Figure 23 - Utilization (hierarchy view): 161 registers, 206 LUTs, 34 IOB, 1 BUFGCTRL

פליפ-פלופים: יש הפרש של 2 בין ההערכה לבין התוצאה, והוא מוסבר בשני שינויים של הסינתזה. הקידוד מחדש של TX_FSM ל-one-hot הוסיף 3 פליפ-פלופים: במכונה יש 61 במקום 58, וזה בדיוק המספר שמופיע בדוח עבור TX_FSM_u. שנית, הביט העליון של byte_cnt, ביט 16, הוסר. הוא משמש רק לספירה עד 65,536, אבל אף פעם לא מגיע ל-8 הביטים שנכנסים לתצוגה, גם בהזזה הגדולה ביותר (8), שלוקחת את ביטים 15 עד 8. ביט בלי עומס נמחק בסינתזה. 159 + 3 - 1 = 161. זה אותו חיתוך שהלינטר דיווח עליו ב-ASSIGN-6.

IOB: 34 הפינים הם 15 כניסות (השעון, הריסט, הכפתור ו-12 מתגים) ו-19 יציאות. ארבעת המתגים שלא בשימוש לא קיבלו IOB, כי הסינתזה הסירה אותם. ה-BUFGCTRL היחיד הוא הבאפר של השעון, כי בתכן יש שעון אחד בלבד.

היררכיה: בתצוגה ההיררכית מופיעים רק המודול העליון ו-TX_FSM. שאר המודולים קטנים, והסינתזה מיזגה אותם לתוך המודול העליון כדי לייעל את הלוגיקה. כדי לראות כל מודול בנפרד, אפשר להריץ סינתזה עם flatten_hierarchy = none.

Step 8 - Summary

המערכת עובדת על הלוח כנדרש: לחיצה קצרה נועלת את הערכים ומתחילה שידור, הריבוע מגיע למחשב בפורמט הנכון, עם רווחים, ירידות שורה והשהיות, לחיצה באמצע שידור מתעלמת, וריסט עוצר הכול מיד. קבצי הלוג של PuTTY תואמים בדיוק, עד הבית, למספר שחושב מראש. הלינטר נקי אחרי שני ויתורים מנומקים, ומספר הפליפ-פלופים בפועל מוסבר במלואו מול ההערכה.

תהליך התכן: עבדתי לפי אותו סדר כמו במעבדה הקודמת: הבנת הדרישות, דיאגרמת בלוקים, תכנון ה-FSM ומתוכו המונים, פירוט הסיגנלים, ורק אז כתיבת קוד. המעבר הכי עדין, סוף שורה, תוכנן מראש בטבלת הדוגמה, ולכן ההחלטה על הסדר בין ההשהיה, CR ו-LF ומונה השורות לא הגיעה מניסוי וטעייה.

אימות: כל בלוק נבדק בסימולציה לבד לפני החיבור, ואחר כך המערכת כולה. הפרמטרים אפשרו להריץ סימולציה מהירה בלי לשנות את הקוד שנצרב. על הלוח, השמירה לקובץ לוג אפשרה בדיקה מדויקת של הפלט, גם כשהטרמינל לא יכול להציג אותו.

שיפור מהמעבדה הקודמת: המגבלה שציינתי במעבדה 3 תוקנה: הריסט עובר עכשיו דרך reset synchronizer.

שיפורים אפשריים: אפשר להוסיף לקובץ האילוצים את הגדרות CFGBVS ו-CONFIG_VOLTAGE כדי לנקות את אזהרת ה-DRC. אפשר גם לצמצם את byte_cnt ל-16 ביט, כי הביט העליון ממילא לא משפיע על התצוגה, וכך להעלים גם את אזהרת הלינטר. בנוסף, אחרי ריסט באמצע שורה השידור הבא מתחיל באותה שורה בטרמינל. אם רוצים, אפשר לשלוח CR ו-LF בתחילת כל שידור חדש.